INDIAN INSTITUTE OF INFORMATION TECHNOLOGY KALYANI

Department of Computer Science and Engineering

Software Engineering Project Assignment

MediFlow

Hospital Management System

Assignment 1 · Hospital Management System

Software Design Document (SDD)

GROUP 11

Sl.	Name of the member	Registration number
1	Akash Chakraborty	CSE/23011/1059
2	Anipriyo Mandal	CSE/23017/1065
3	Arpan Mandal	CSE/23027/1075
4	Himanshu Patel	CSE/23052/1100
5	Pranjali Sirari	CSE/23072/1120

Live demo at mediflow-hms-6vw1.onrender.com

Source code at github.com/anipriyo/mediflow-hms

Submitted through Google Classroom · October 2026

Group Details

Table 1. Group and assignment details

Field	Value
Group number	11
Assignment number and title	Assignment 1, Hospital Management System
Document	Software Design Document (SDD) (document 2 of 5)
Project name	MediFlow Hospital Management System
Problem statement (brief)	Develop a system to manage patients, doctors, appointments, prescriptions, billing and medical records.
Companion documents	Software Requirements Specification (SRS), UML Diagrams, Software Test Plan, Software Test Report
Live demo	mediflow-hms-6vw1.onrender.com
Source code	github.com/anipriyo/mediflow-hms

Table 2. Group members (names and registration numbers)

Sl.	Name	Registration number
1	Akash Chakraborty	CSE/23011/1059
2	Anipriyo Mandal	CSE/23017/1065
3	Arpan Mandal	CSE/23027/1075
4	Himanshu Patel	CSE/23052/1100
5	Pranjali Sirari	CSE/23072/1120

Document control

Table 3. Revision history

Version	Date	Description
0.1	15-Oct-2026	Project selected (Assignment 1, Hospital Management System)
0.2	18-Oct-2026	Scope, user roles and feature set agreed (core, hospital operations, distinctive features)
0.5	21-Oct-2026	Requirements baselined (50 FR, 10 NFR, 12 BR), architecture and UML diagrams
0.9	21-Oct-2026	Prototype implemented, unit, integration, system and acceptance tests executed
0.95	22-Oct-2026	Robustness / edge-case testing, additional defects fixed and regression-tested
1.0	22-Oct-2026	Final report for submission
1.1	22-Oct-2026	Prototype deployed online (Render), live demo link added
1.2	25-Oct-2026	Issued as a separate document (Software Design Document (SDD)), one of five

Summary

MediFlow is a web-based Hospital Management System that manages **patients, doctors, appointments, prescriptions, billing and medical records**, extended with laboratory, pharmacy and in-patient (bed and admission) management. Beyond the core requirements, it introduces a **smart queue** with tokens and waiting-time estimates, a **live bed board** with a discharge workflow tied to billing clearance, and **role-based access control with an append-only audit trail** for six staff roles.

This document describes the design that realises the SRS. It covers the layered architecture and deployment, the thirteen modules, the database design, the class, use case, sequence, state and activity diagrams with use case specifications, the security design and the user interface.

> **Live demo.** The prototype runs online at mediflow-hms-6vw1.onrender.com and opens in any browser with no installation. Sign in with one of the demo accounts (admin, reception, dr.sharma, lab, pharmacy and billing, all with the password Mediflow@123). It uses a free cloud plan, so the first visit after 15 idle minutes takes about a minute to load, and the demo data is reset to fresh records dated that day whenever the server restarts.
>
> **Source code and running it locally.** The full source code is at github.com/anipriyo/mediflow-hms. Install Node.js 20 or later, run `npm install` and then `npm run dev` in the project folder, and open localhost port 5173 in a browser.

Table of Contents

Group Details..... 2
Document control..... 2
Summary..... 3
Table of Contents..... 4
SOFTWARE DESIGN DOCUMENT (SDD)..... 5
1. System Architecture..... 5
1.1 Design decisions and rationale..... 6
1.2 Deployment view..... 7
2. Module / Component Description..... 7
2.1 Cross-cutting components..... 8
3. Database Design (optional)..... 8
4. Class Diagram..... 9
5. Use Case Diagram..... 12
5.1 Use case specifications..... 14
6. Sequence Diagrams..... 16
7. State Diagrams..... 20
7.1 Activity view of the outpatient journey..... 22
8. Security Design..... 23
9. User Interface Design..... 24

SOFTWARE DESIGN DOCUMENT (SDD)

> The design follows the IEEE Std 1016-2009 viewpoints, namely context and composition (architecture, modules), logical (class diagrams), interaction (sequence diagrams), state dynamics (state diagrams), information (data model) and interface (UI and API). All diagrams are UML 2.5 and were produced with PlantUML from the source files in docs/uml/.

1. System Architecture

MediFlow uses a **three-tier, layered client to server architecture**. The presentation tier is a React single-page application running in the browser. The application tier is a Node.js/Express server exposing a REST API. The data tier is an embedded SQLite database. Inside the server, responsibilities are separated into four layers, and each layer may only call the layer below it.

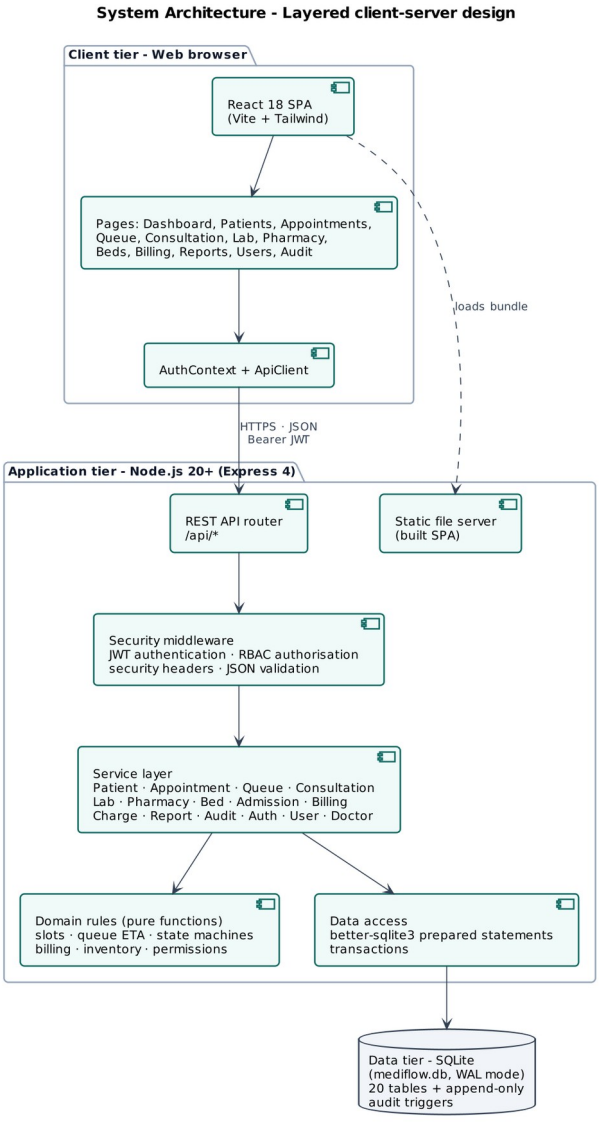


Figure 1. Layered system architecture

Table 4. Architectural layers

Layer	Responsibility	Implementation
Presentation	Screens, navigation by role, forms with client-side validation, charts, printing	client/src, 18 page components, shared UI kit, AuthContext, ApiClient

Layer	Responsibility	Implementation
API / controller	HTTP routing, JSON parsing, authentication, authorisation, error mapping to status codes	server/routes/index.js, server/middleware/auth.js, server/app.js
Service	Use-case logic and transactions, orchestrates rules, data access and audit	server/services/*.js (15 services)
Domain rules	Pure, side-effect-free business rules. Slots, conflicts, queue ETA, state machines, billing, inventory, permissions	server/domain/*.js (unit-tested in isolation)
Data access	Connection, schema, prepared statements, transactions, constraints and triggers	server/db/connection.js, schema.sql, seed.js

1.1 Design decisions and rationale

Table 5. Key design decisions

Decision	Rationale
Web SPA + REST API	Works on any device with a browser, no installation at workstations, the API can later serve a mobile app or integrations.
Business rules as pure functions	Rules such as slot validation, ETA and bed-day allocation are deterministic and testable without a database (NFR-06). A frozen clock makes time-based rules reproducible.
SQLite with WAL and transactions	Zero-administration database suitable for one hospital server and for marking/demonstration. ACID transactions guarantee multi-step operations (NFR-04).
Constraints in the schema as well as in code	Defence in depth. Unique partial indexes stop double bed allocation and double active admissions even under concurrent requests.
JWT + server-side sessions	Stateless verification with the ability to revoke on logout or deactivation (FR-04, FR-06).
Charges as an intermediate ledger	Clinical modules only record billable events. Billing turns them into invoices. This decouples modules and prevents missed charges.
Append-only audit log	Accountability required by O7, enforced by database triggers so even direct SQL cannot alter history.

1.2 Deployment view

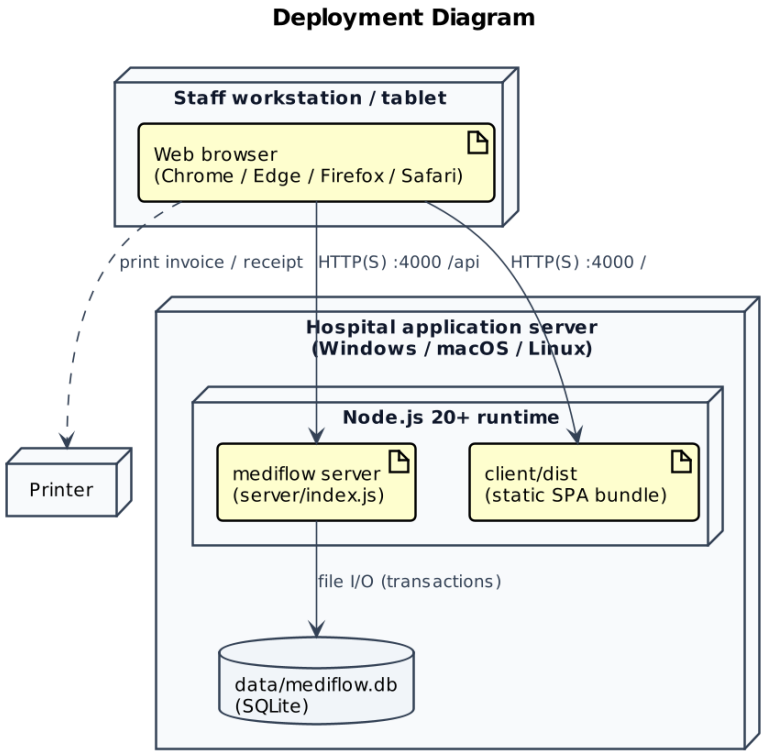


Figure 2. Deployment diagram

The public demonstration uses the same single-server design. One Node.js web service on Render's cloud serves both the built React app and the REST API with an embedded SQLite database, at mediflow-hms-6vw1.onrender.com. Render builds it from the GitHub repository (`render.yaml`) and redeploys on every push. HTTPS is terminated by Render's proxy, matching the reverse-proxy setup recommended for production.

The prototype consists of about 3,577 lines of server code in 39 files, 5,497 lines of client code in 29 files and 2,582 lines of automated tests.

2. Module / Component Description

The system is decomposed into thirteen functional modules. Each module owns its screens, service, rules and tables, and communicates with other modules only through service calls and shared identifiers (patient, appointment, admission, charge).

Table 6. Module / component description

Module	Responsibility	UI pages	Services / rules	Tables	FRs
Authentication & Access Control	Login/logout, JWT sessions, account lock-out, role permissions, route guards	Login, Layout (role menu)	AuthService, auth middleware, authRules, permissions	users, sessions	FR-01 to FR-04
User Administration	Create users, (de)activate, reset passwords, role-permission matrix view	Users & Roles	UserService	users	FR-05, FR-06
Patient Management	Registration with unique ID, validation, duplicate detection, search, edit, history	Patients, Patient detail	PatientService, patientRules	patients	FR-07 to FR-12
Doctor & Department	Doctor profiles, schedules, fees, departments, free-slot calculation	Doctors	DoctorService, appointmentRules	doctors, departments	FR-13 to FR-15
Appointments	Book, reschedule, cancel, no-show, filters, 4-step booking wizard	Appointments	AppointmentService, appointmentRules	appointments	FR-15 to FR-20

Module	Responsibility	UI pages	Services / rules	Tables	FRs
Smart Queue	Check-in tokens, queue board, ETA, urgent priority, call next	Queue Board	QueueService, queueRules	appointments, consultations	FR-21 to FR-25
Consultation & Records	Vitals, diagnosis, notes, follow-up, prescription builder, lab ordering, history	Consultation	ConsultationService, ChargeService	consultations, prescriptions, prescription_items	FR-26 to FR-30
Laboratory	Worklist, sample collection, result entry, abnormal flag, cancellation	Laboratory	LabService, labRules	lab_tests, lab_orders	FR-31 to FR-34
Pharmacy & Inventory	Medicine master, restock, alerts, dispensing	Pharmacy	PharmacyService, inventoryRules	medicines, prescriptions	FR-35 to FR-37
Admission, Bed & Discharge	Bed board, admit, transfer, bed status, discharge request, clearance, override	Beds & Admissions, Admission detail	BedService, AdmissionService, bedStateMachine, admissionRules	wards, beds, admissions, bed_stays	FR-38 to FR-43
Billing & Payments	Pending charges, invoices, discounts, payments, receipts, printing	Billing, Invoice detail, Print views	BillingService, ChargeService, billingRules	charges, invoices, payments	FR-44 to FR-47
Dashboard & Reports	Role-aware KPIs, 7-day trends, management reports, CSV export	Dashboard, Reports	ReportService	(read-only aggregates)	FR-48, FR-49
Audit Trail	Append-only log of every state change. Search & filter	Audit Trail	AuditService	audit_logs (+ triggers)	FR-50

2.1 Cross-cutting components

- **Authentication middleware** verifies the JWT, checks that the session is not revoked or expired and that the account is active, and attaches the user (with role and doctor id) to the request.
- **Authorisation**, `authorize(permission…)` checks the role-permission matrix (Section 8). The UI hides menus the role cannot use, but the server is the authority.
- **Validation**, zod schemas and domain validators return HTTP 400 with field-level messages that the UI shows next to the inputs.
- **Error handling**, a central handler maps errors to consistent JSON. 400 validation, 401 unauthenticated, 403 forbidden, 404 not found, 409 business-rule conflict, 423 account locked, 500 unexpected.
- **ChargeService**, a shared ledger used by Consultation, Laboratory, Pharmacy and Admission modules to create billable events.
- **AuditService**, called by every service after a successful state change. Scrubs credential-like fields.
- **Clock**, a single time source that tests can freeze.

3. Database Design (optional)

Database design is not required by the assignment. It is included briefly because the class diagram maps directly to the relational schema. The data model has 20 tables in third normal form. Status columns are constrained by CHECK clauses that mirror the state diagrams in Section 7.

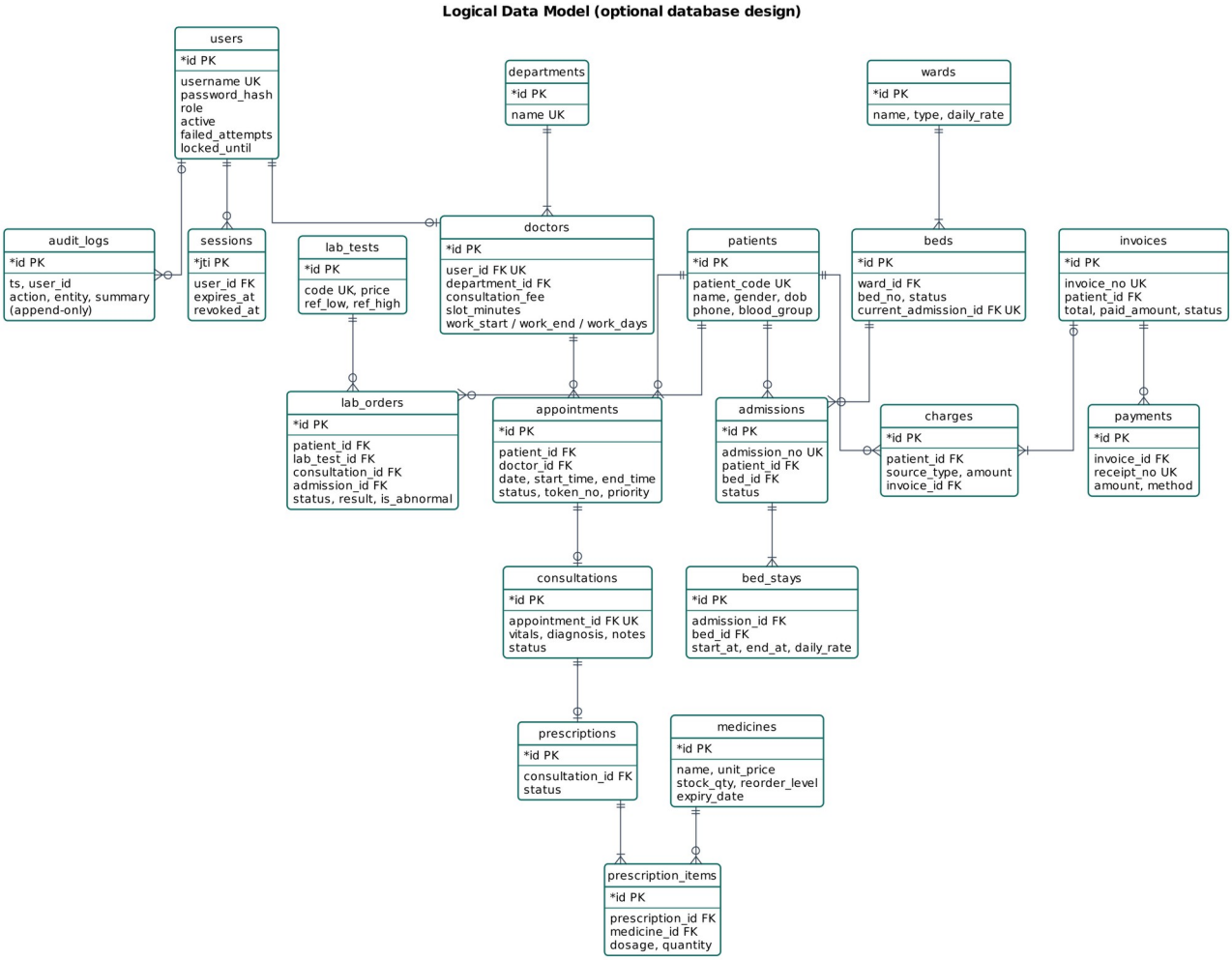


Figure 3. Logical data model (crow's-foot notation)

4. Class Diagram

The **domain class diagram** (Figure below) shows the main entities, their attributes, operations and associations with multiplicities. A patient books many appointments. Each appointment has at most one consultation, which may produce one prescription (composed of items referencing medicines) and several lab orders. Wards are composed of beds. An admission is composed of one or more bed stays (one per bed occupied), which is how transfers and bed-day billing are modelled. Charges aggregate into invoices, which own payments.

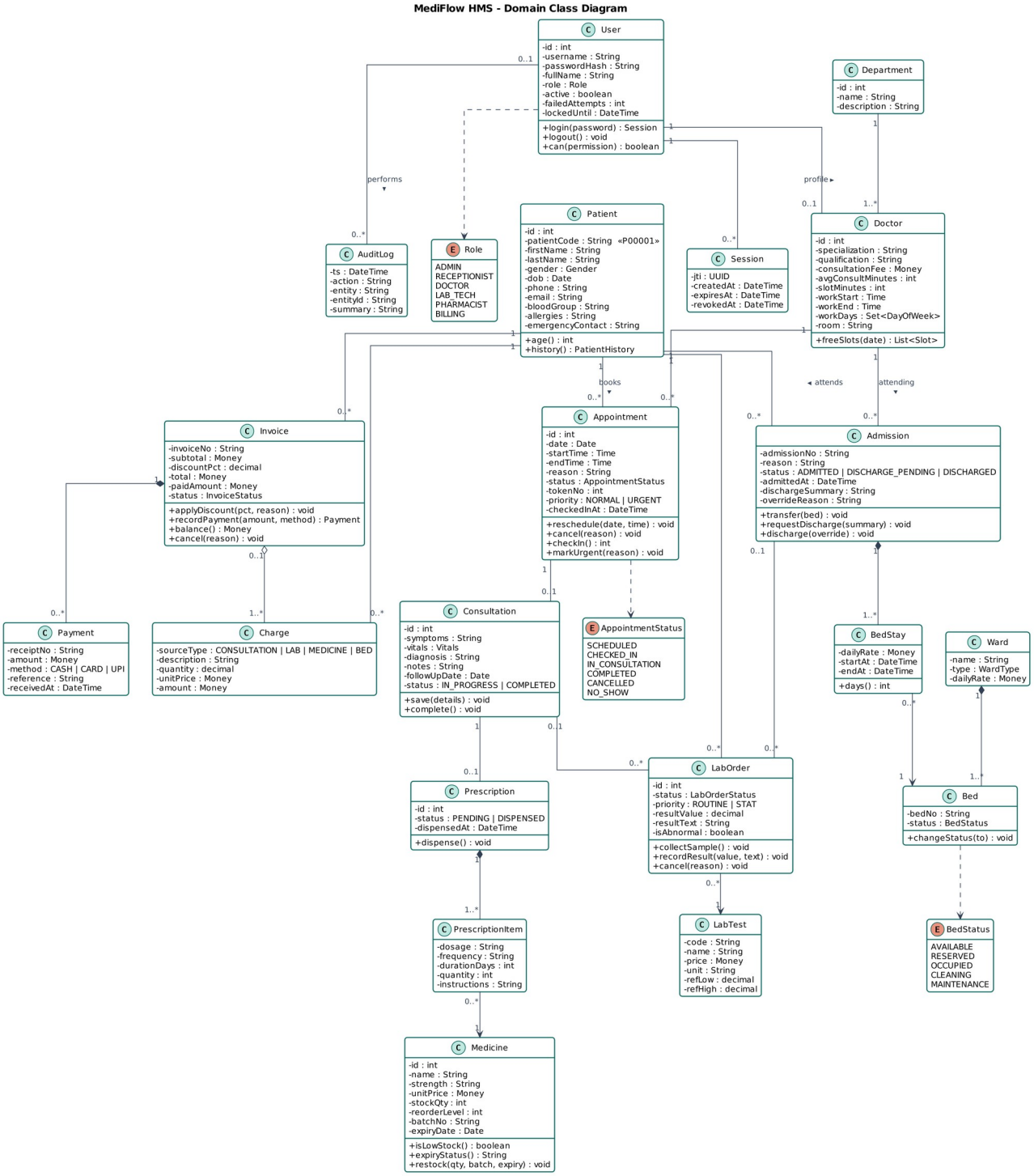


Figure 4. Domain class diagram

The **design class diagram** shows how the implementation realises the domain. Controllers delegate to services, services use pure rule modules and the data-access layer, and cross-cutting services (Charge, Audit) are shared.

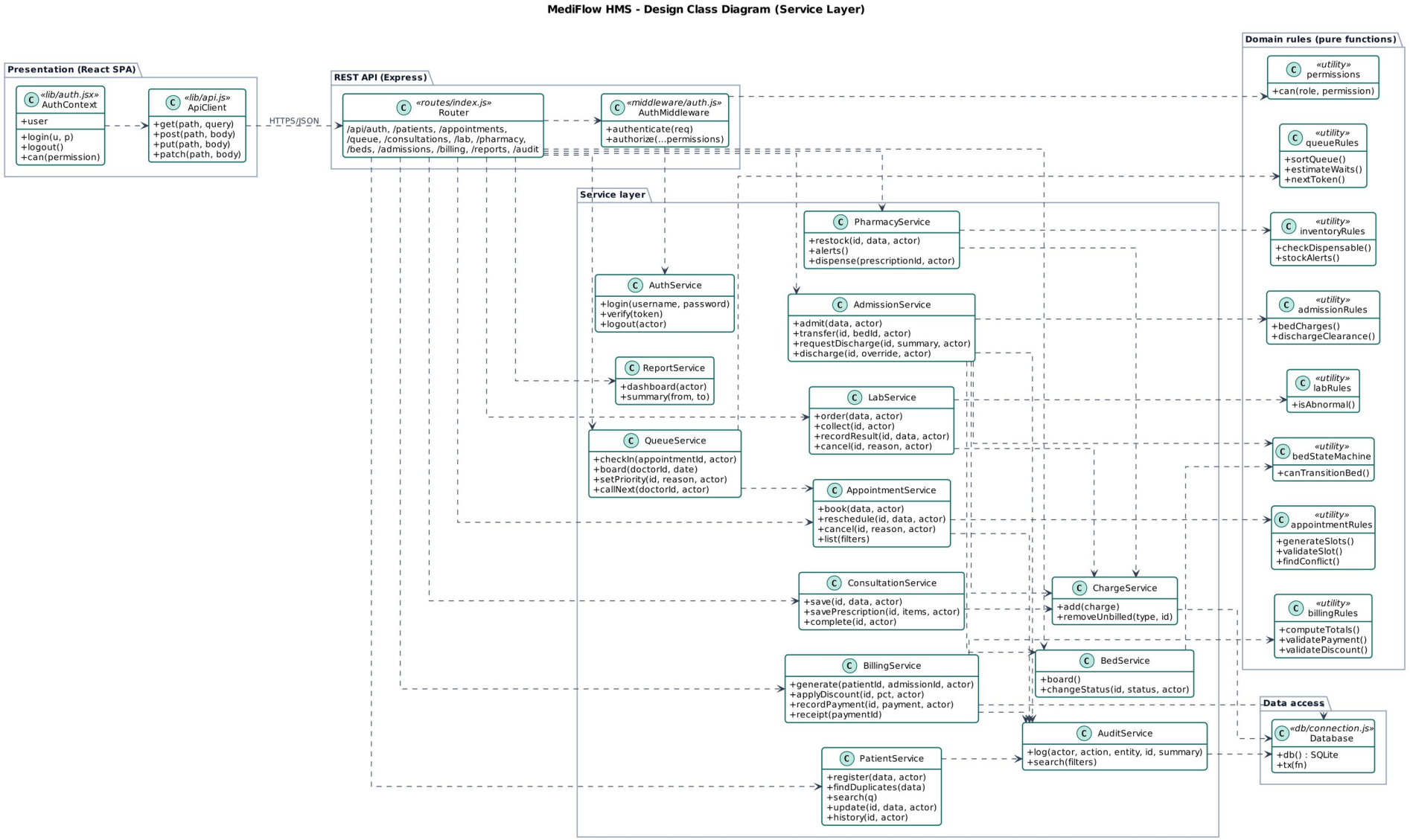


Figure 5. Design class diagram (service layer and dependencies)

5. Use Case Diagram

Because a single diagram with six actors and forty use cases is unreadable, the use-case model is split into four diagrams. These are an overview (SRS, Section 4.3), outpatient care, inpatient care & billing, and administration & access control. «include» marks behaviour that always happens as part of the base use case, while «extend» marks optional behaviour.

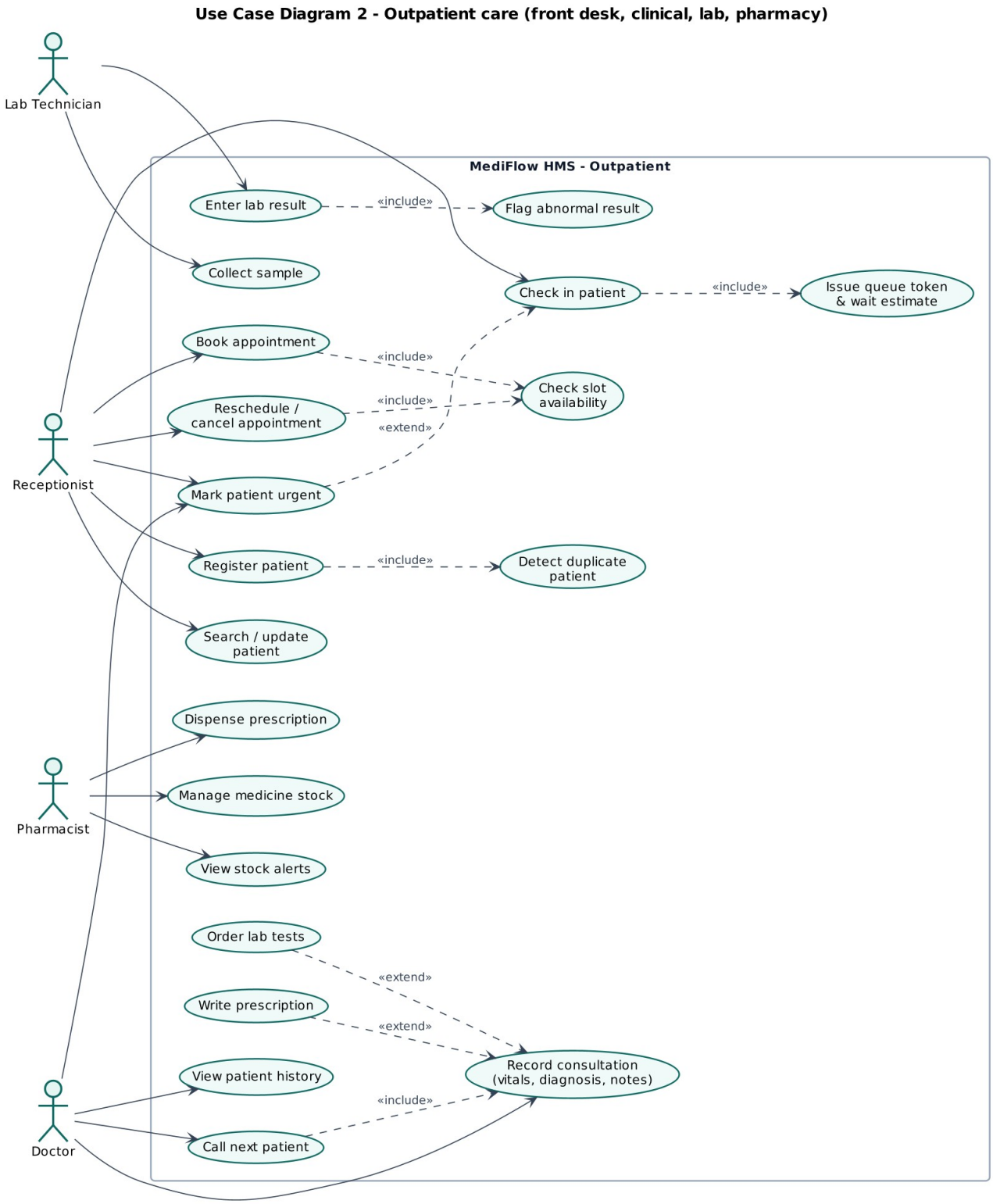


Figure 6. Use case diagram (outpatient care)

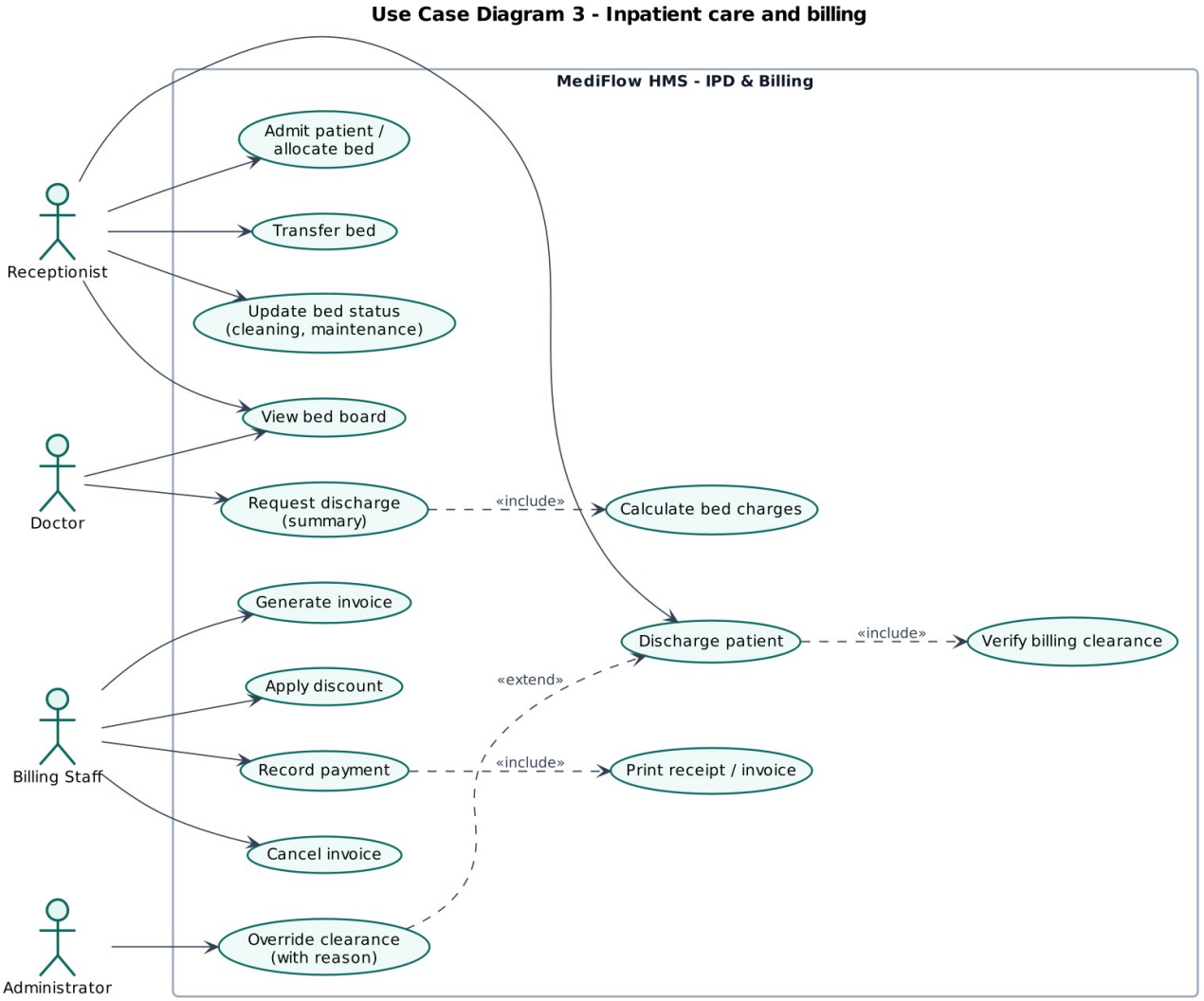


Figure 7. Use case diagram (inpatient care and billing)

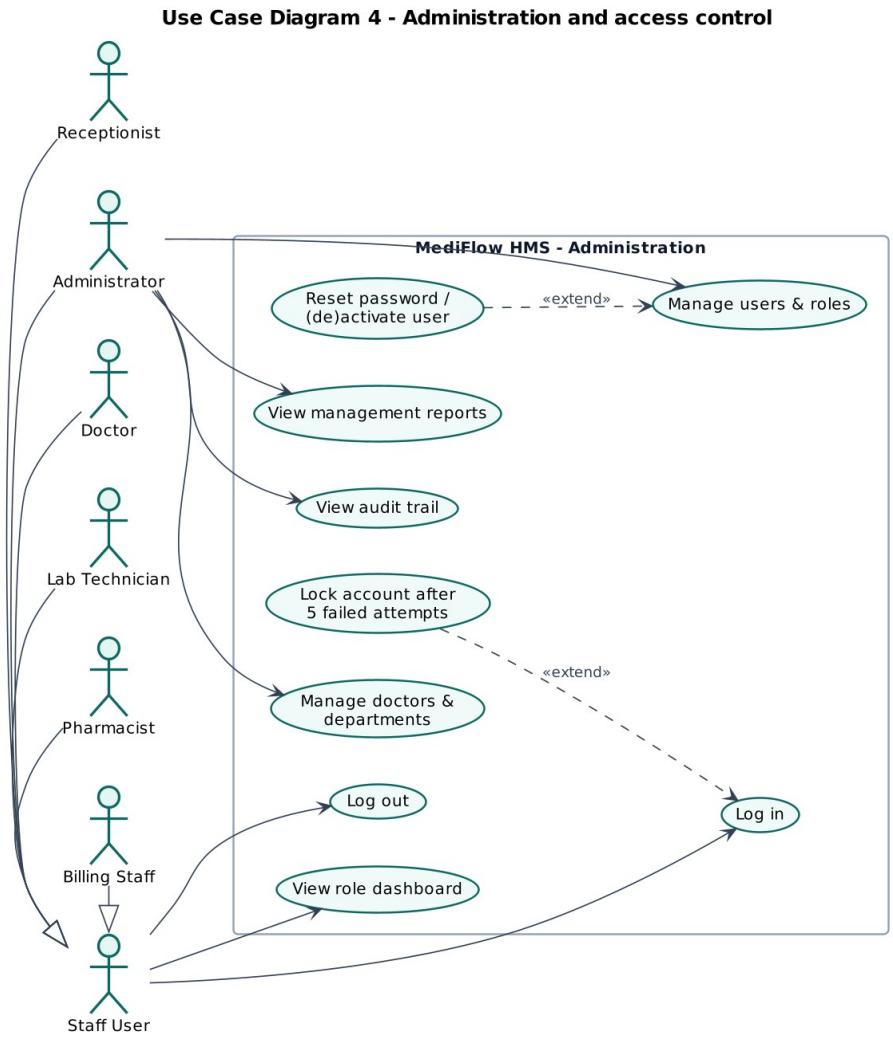


Figure 8. Use case diagram (administration and access control)

5.1 Use case specifications

Table 7. Use case specification UC-01 (Book appointment)

UC-01	Book appointment
Primary actor(s)	Receptionist
Requirements	FR-15 to FR-17
Pre-condition	Receptionist is logged in, patient is registered, doctor works on the chosen date.
Main success scenario	1. Receptionist opens Appointments → Book appointment. 2. Selects the patient (search by name / ID / phone). 3. Chooses department and doctor, the system shows fee, room and hours. 4. Chooses a date, the system displays all slots marked free, booked or past. 5. Selects a free slot, enters the reason and confirms. 6. System validates the slot, checks doctor and patient overlaps, saves the appointment as SCHEDULED and writes an audit entry.
Alternative flows	5a. Slot was taken meanwhile → HTTP 409 DOCTOR_SLOT_TAKEN. The wizard refreshes the slots. 5b. Patient already has an overlapping appointment → HTTP 409 PATIENT_DOUBLE_BOOKED. 4a. Date is a non-working day → no slots are offered.
Post-condition	Appointment exists in SCHEDULED state and blocks the slot.

Table 8. Use case specification UC-02 (Check in patient and manage queue)

UC-02	Check in patient and manage queue
Primary actor(s)	Receptionist (primary), Doctor
Requirements	FR-21 to FR-25
Pre-condition	Appointment is SCHEDULED for today.
Main success scenario	1. Receptionist clicks Check in for the appointment. 2. System assigns the next token for the doctor and sets CHECKED_IN. 3. Queue board shows the patient with position and ETA. 4. Doctor clicks Call next patient. 5. System picks the first patient (URGENT first, then by check-in time), sets IN_CONSULTATION and opens a consultation record.
Alternative flows	2a. Appointment is not for today → HTTP 400. 3a. Patient deteriorates → receptionist/doctor marks URGENT with a mandatory reason (audited). Patient moves to the front. 4a. Doctor already has a consultation in progress → HTTP 409.
Post-condition	Patient is in consultation, others' ETA is recalculated.

Table 9. Use case specification UC-03 (Conduct consultation)

UC-03	Conduct consultation
Primary actor(s)	Doctor
Requirements	FR-26 to FR-29
Pre-condition	A consultation is IN_PROGRESS for the doctor.
Main success scenario	1. Doctor records vitals, symptoms, diagnosis, notes and follow-up date and saves. 2. Adds medicines (dosage, frequency, duration, quantity auto-calculated) and saves the prescription. 3. Orders lab tests (ROUTINE or STAT). Charges are created. 4. Clicks Complete consultation. 5. System checks the diagnosis, sets the consultation and appointment COMPLETED and adds the consultation-fee charge.
Alternative flows	1a. Invalid vitals (e.g. BP “120-80”) → field error. 2a. Quantity ≤ 0 → validation error. 4a. Diagnosis missing → HTTP 400. Consultation stays open.
Post-condition	Visit completed, prescription PENDING at pharmacy, lab orders ORDERED, charges waiting for billing.

Table 10. Use case specification UC-04 (Discharge patient)

UC-04	Discharge patient
Primary actor(s)	Doctor, Receptionist, Billing Staff, Administrator
Requirements	FR-42, FR-43
Pre-condition	Patient is ADMITTED.
Main success scenario	1. Doctor requests discharge with a summary (≥ 10 characters). 2. System closes the bed stay, calculates bed-days and adds bed charges. Status DISCHARGE_PENDING. 3. Billing generates the admission invoice and records payment(s) until PAID. 4. Receptionist completes the discharge. 5. System verifies clearance, sets DISCHARGED and moves the bed to CLEANING.
Alternative flows	4a. Charges not invoiced or balance outstanding → HTTP 409 BILLING_NOT_CLEARED. 4b. Emergency transfer. Administrator overrides with a reason. Discharge proceeds and a DISCHARGE_OVERRIDE audit entry records the outstanding amount.
Post-condition	Admission closed, bed in CLEANING until housekeeping marks it AVAILABLE.

Table 11. Use case specification UC-05 (Record payment)

UC-05	Record payment
Primary actor(s)	Billing Staff
Requirements	FR-44 to FR-47
Pre-condition	Invoice exists and is UNPAID or PARTIALLY_PAID.
Main success scenario	1. Billing staff opens the invoice and clicks Record payment. 2. Enters amount (defaults to balance), method and reference (for card/UPI). 3. System validates 0 < amount ≤ balance, stores the payment with a receipt number and updates the status. 4. Receipt is printed.
Alternative flows	3a. Amount exceeds balance → HTTP 400 with the outstanding balance. 3b. Card/UPI without reference → HTTP 400. 3c. Invoice already PAID or CANCELLED → HTTP 400.
Post-condition	Invoice PARTIALLY_PAID or PAID, receipt available for printing.

6. Sequence Diagrams

The sequence diagrams show the interaction between actors, the React UI, the API layer, services, rule modules and the database for the four most important scenarios. Messages are numbered, alt/opt fragments show the error and override paths that are also covered by test cases.

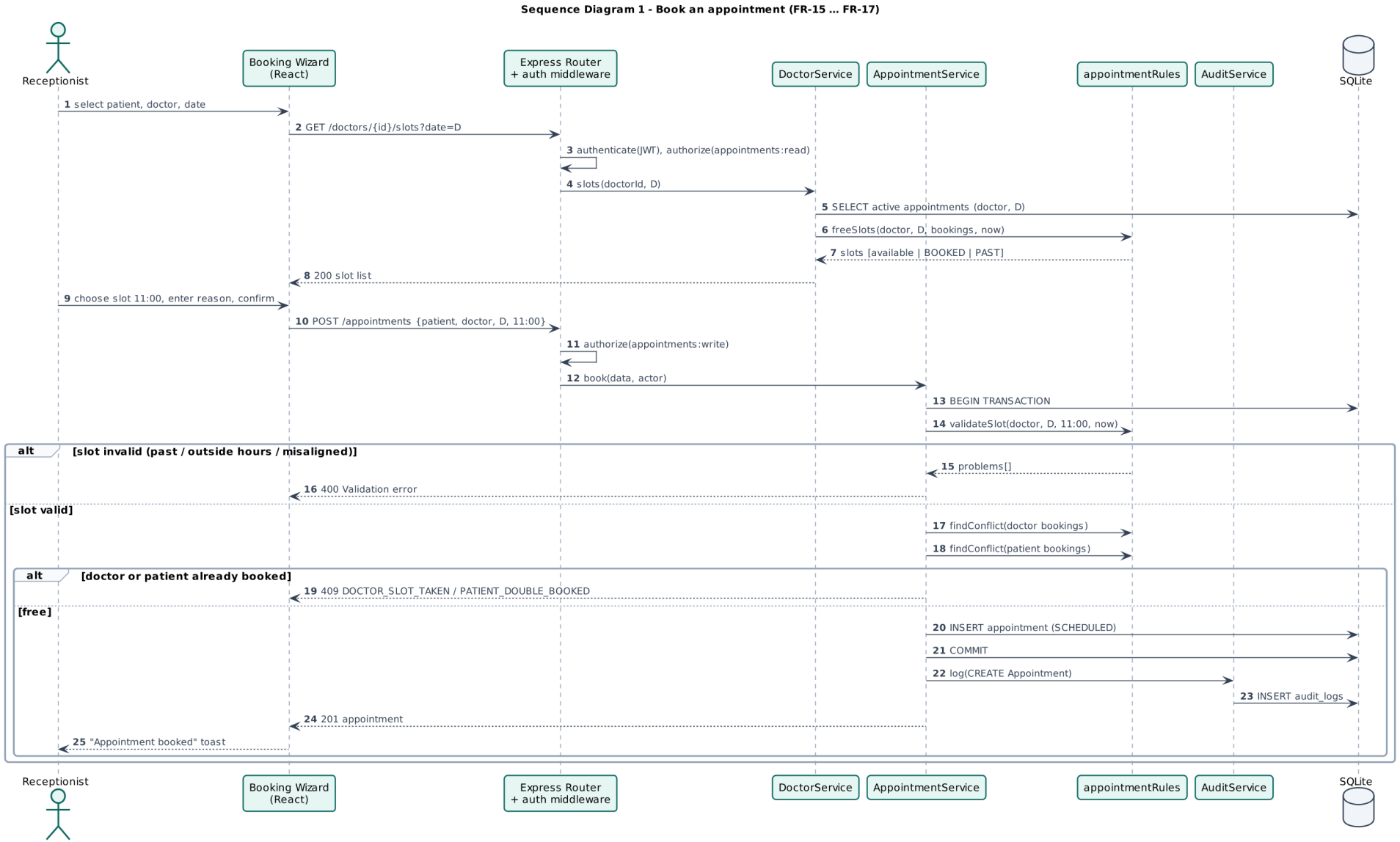


Figure 9. Sequence diagram (book an appointment)

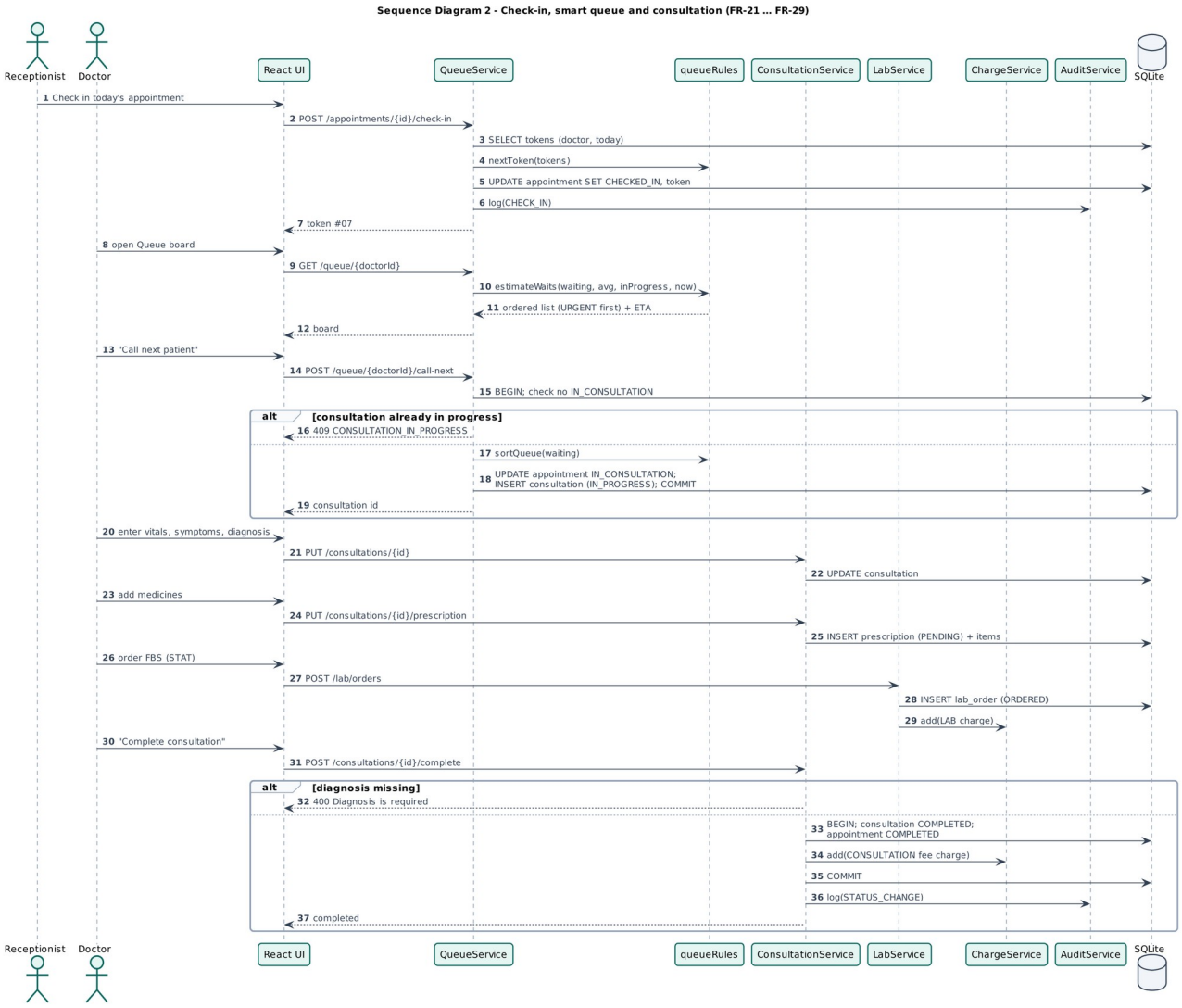


Figure 10. Sequence diagram (check-in, queue and consultation)

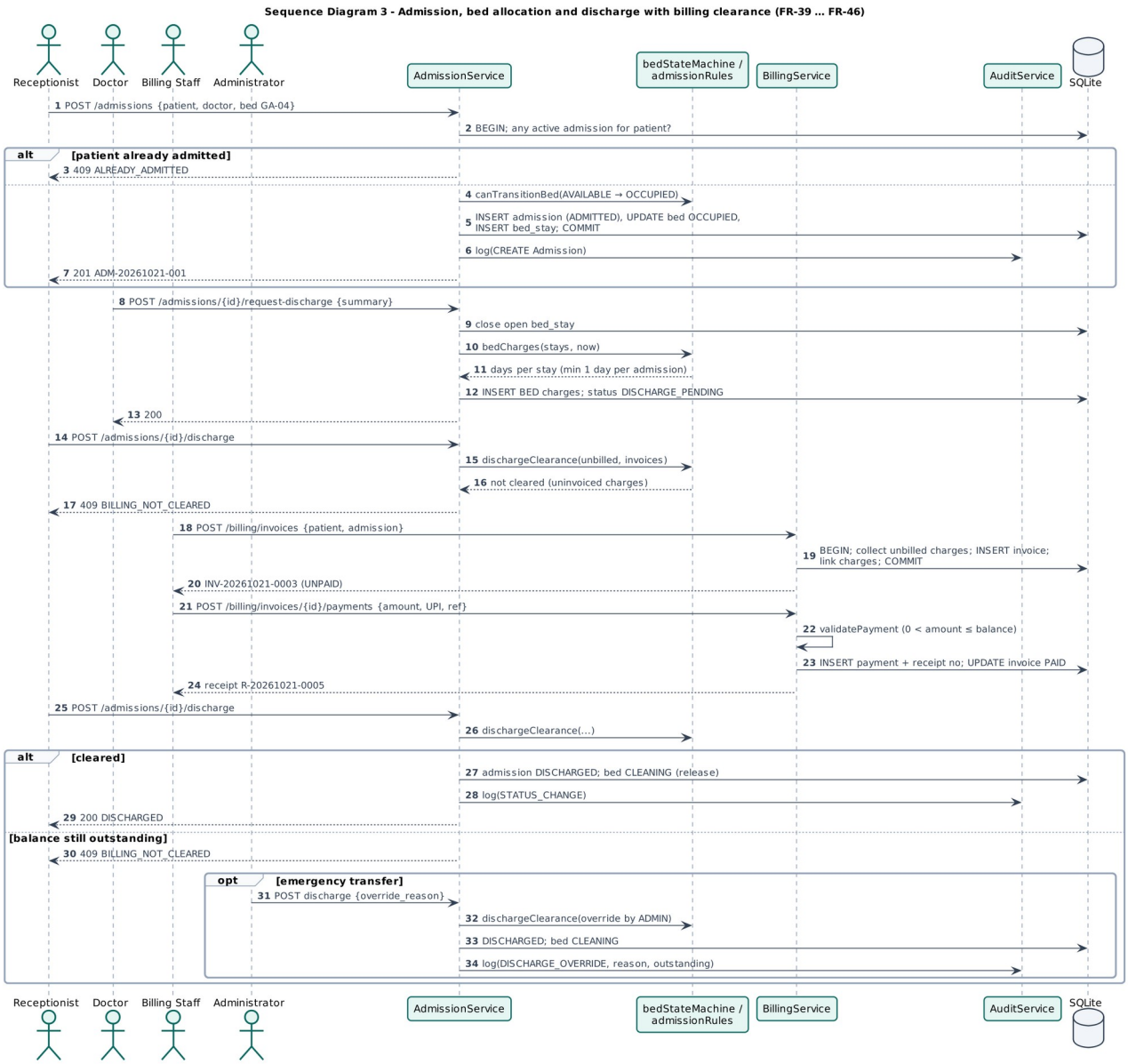


Figure 11. Sequence diagram (admission and discharge with billing clearance)

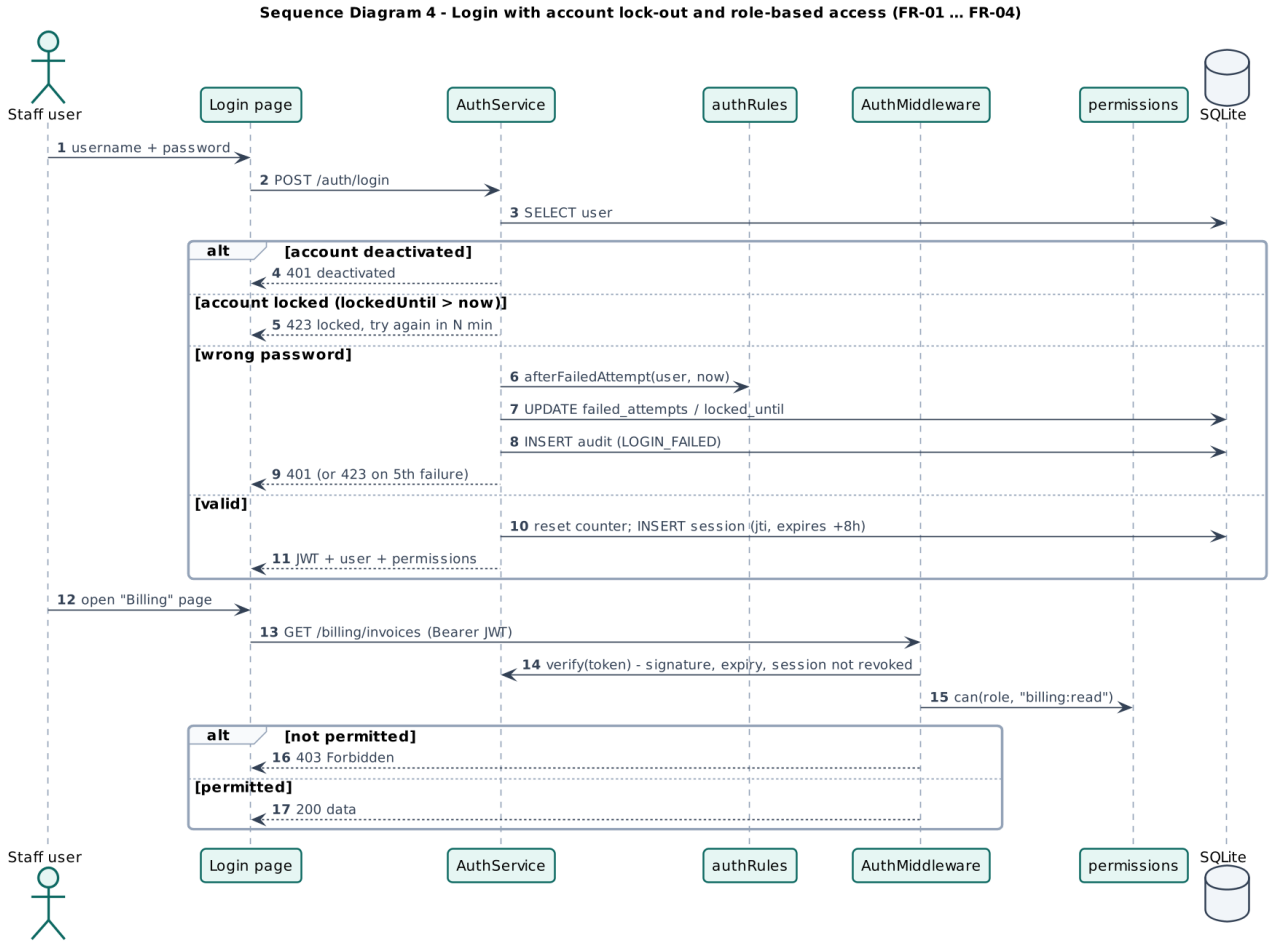


Figure 12. Sequence diagram (login, lock-out and role-based access)

7. State Diagrams

State diagrams are applicable to every entity with a life cycle. Transitions not shown are rejected by the system (HTTP 409), the same transition tables are implemented in `server/domain/*` and verified by unit tests UT-APT-10, UT-IPD-01/02/04, UT-LAB-01 and UT-BIL-06.

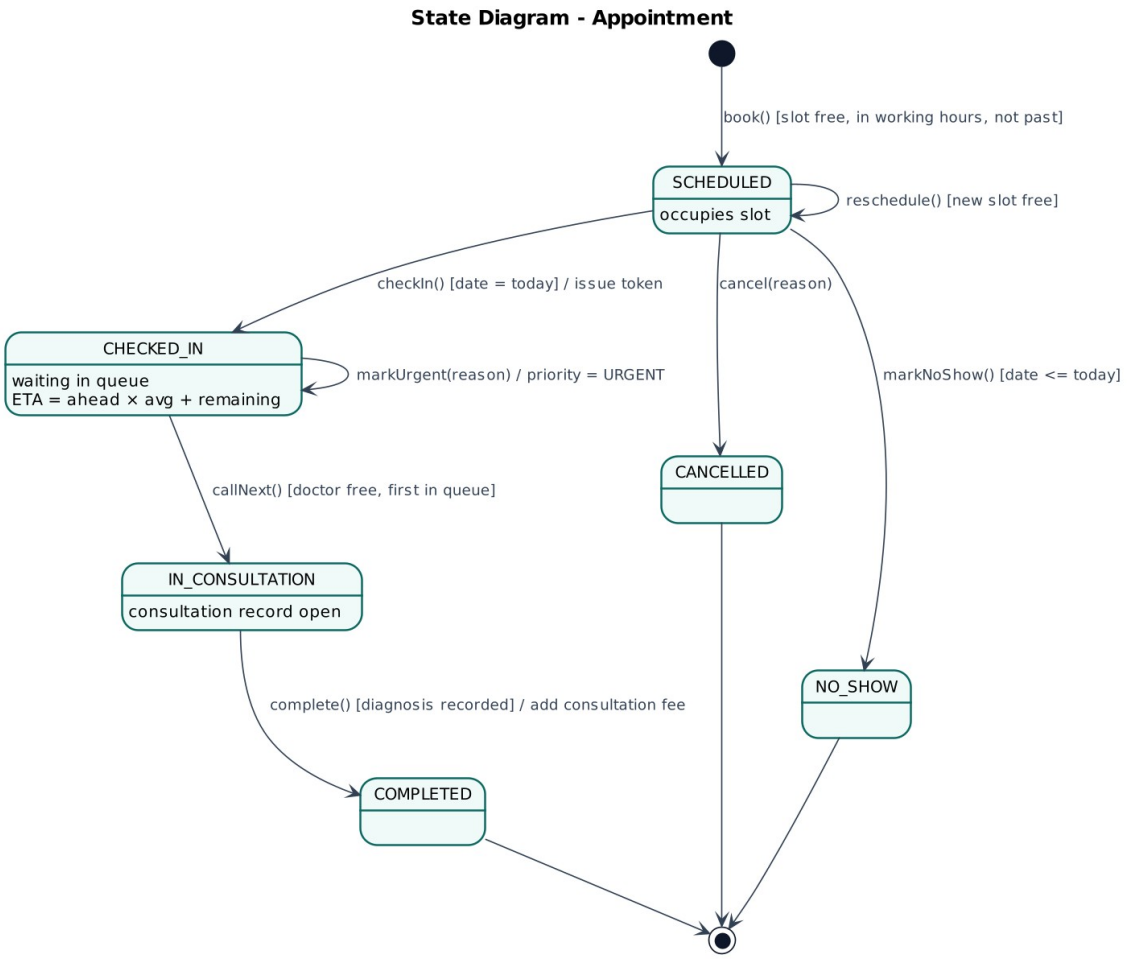


Figure 13. State diagram (Appointment)

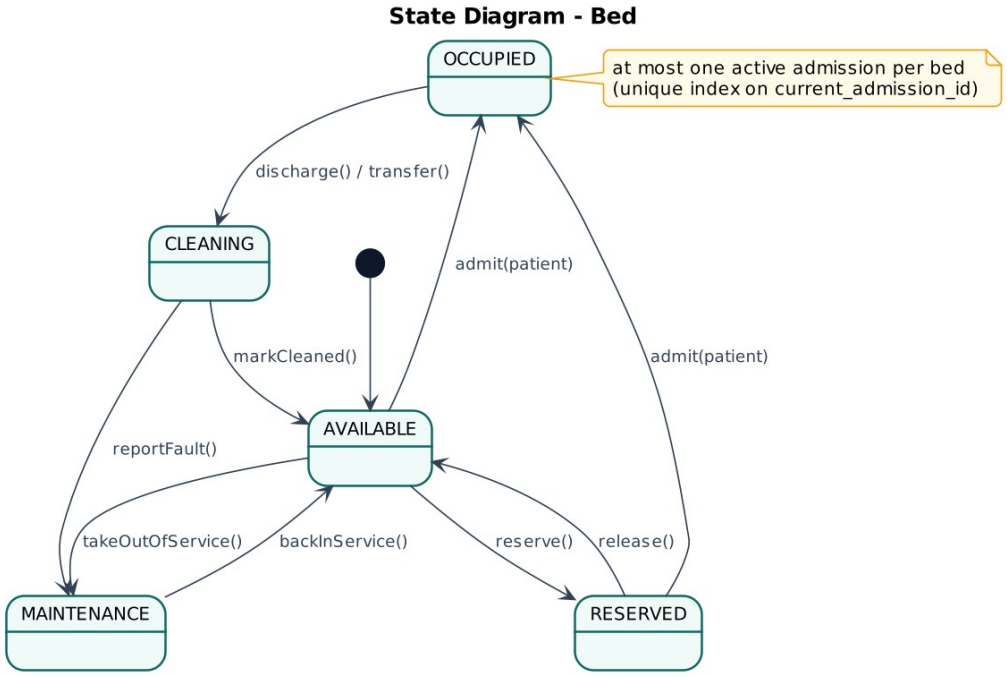


Figure 14. State diagram (Bed)

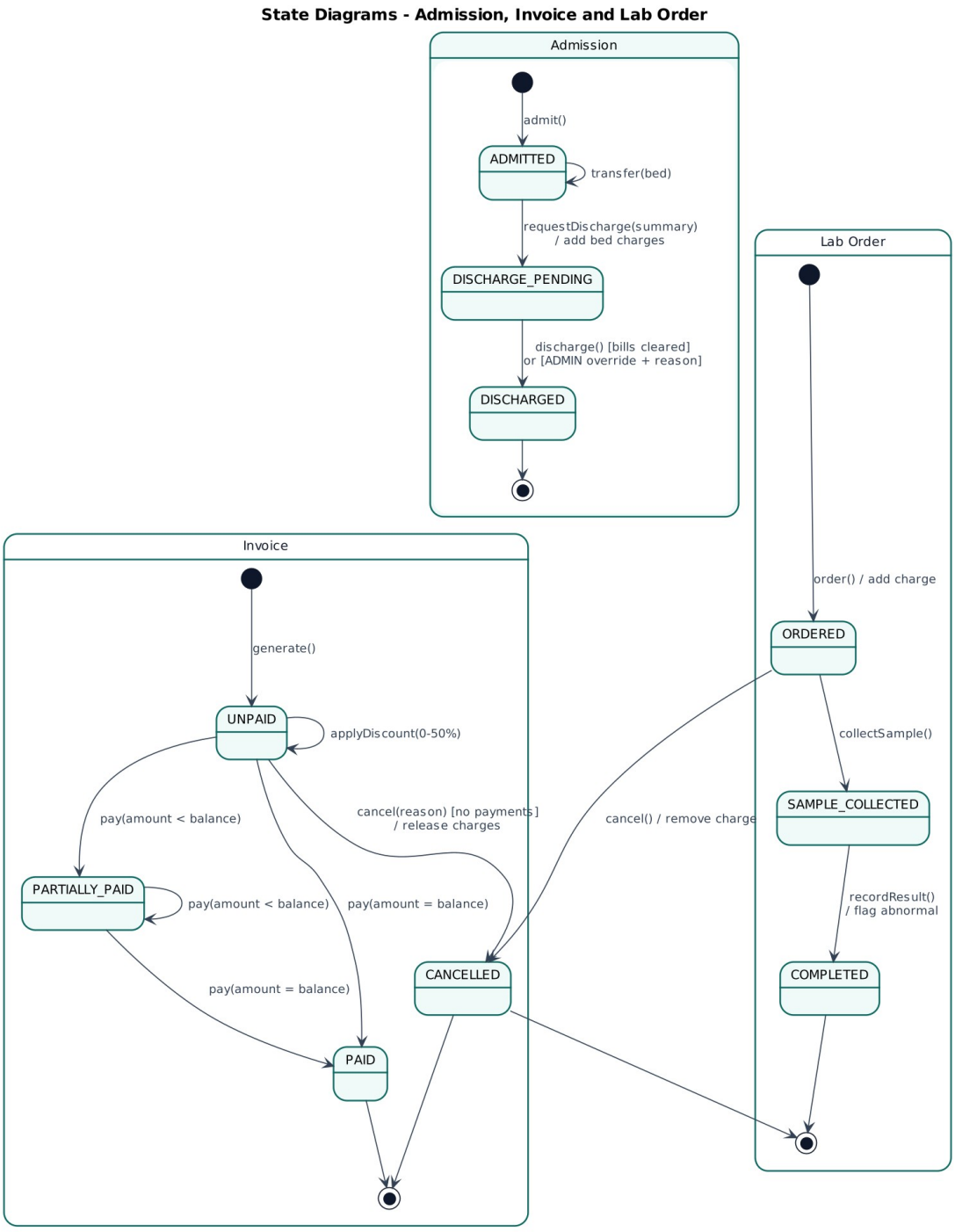


Figure 15. State diagrams (Admission, Invoice and Lab order)

7.1 Activity view of the outpatient journey

The activity diagram complements the state diagrams by showing how the responsibility for one patient visit passes between roles (swimlanes).

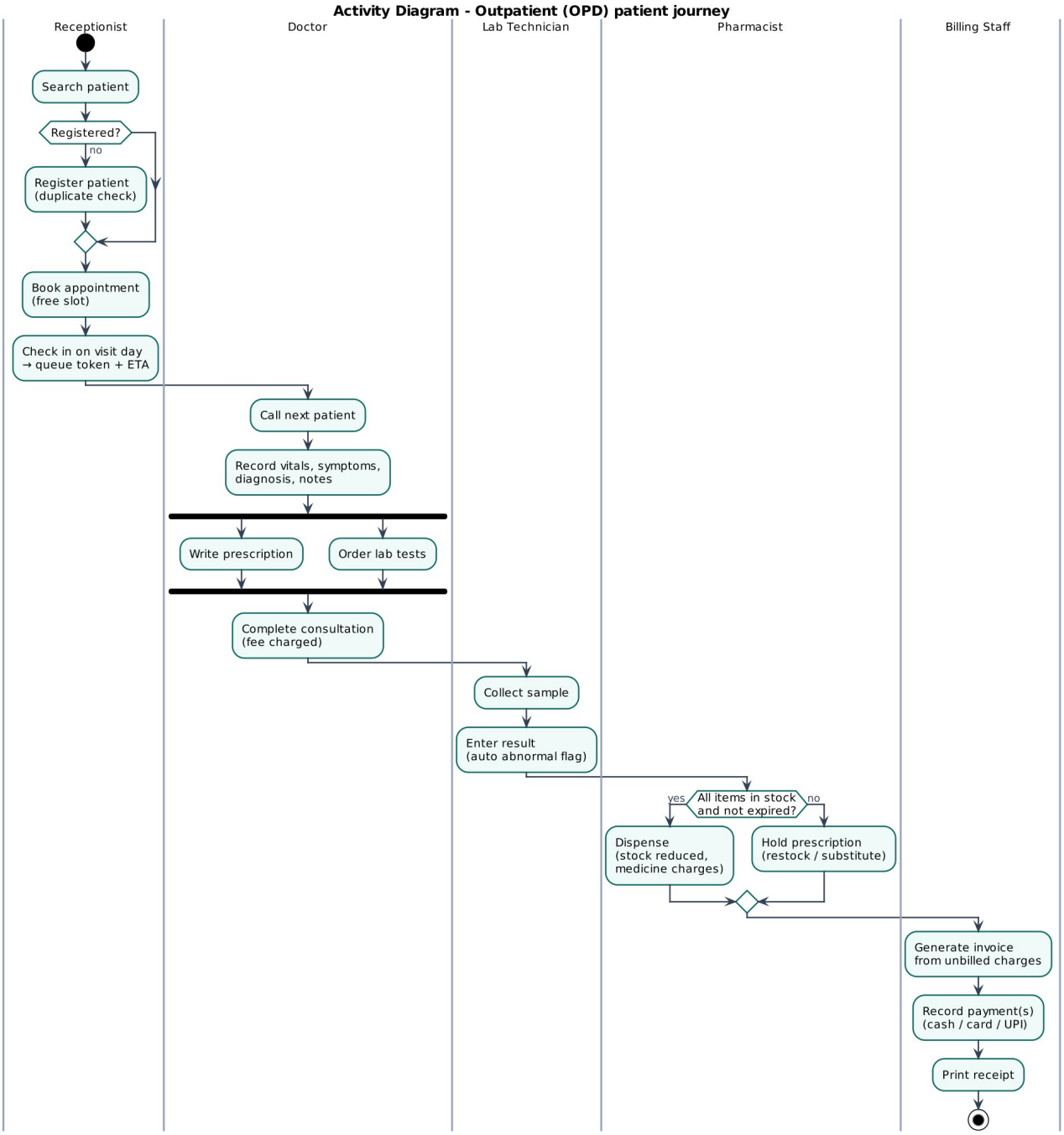


Figure 16. Activity diagram (outpatient journey across roles)

8. Security Design

- **Authentication.** Bcrypt password hashes (cost 10). JWT signed with HS256 valid 8 h, bound to a server-side session that is revoked on logout or deactivation, 5 failed attempts lock the account for 15 minutes. Generic error messages avoid username enumeration.
- **Authorisation.** Least-privilege role-permission matrix enforced on every API route. Doctors may act only on their own queue and consultations. Clinical notes are hidden from non-clinical roles.
- **Input handling.** Schema validation on every write. Parameterised SQL only (no string concatenation). JSON body limit 1 MB. Security headers (nosniff, frame-deny, no-referrer).
- **Integrity and accountability.** Transactions for multi-step updates. Unique and check constraints, append-only audit log with triggers.

Table 12. Role to permission matrix (generated from server/domain/permissions.js)

Permission	Admin	Recep.	Doctor	Lab	Pharm.	Billing
dashboard.read	✓	✓	✓	✓	✓	✓
users.manage	✓	n/a	n/a	n/a	n/a	n/a
doctors.read	✓	✓	✓	✓	✓	✓
doctors.manage	✓	n/a	n/a	n/a	n/a	n/a
patients.read	✓	✓	✓	✓	✓	✓
patients.write	✓	✓	n/a	n/a	n/a	n/a
clinical.read	✓	n/a	✓	n/a	n/a	n/a
appointments.read	✓	✓	✓	n/a	n/a	n/a
appointments.write	✓	✓	n/a	n/a	n/a	n/a
queue.read	✓	✓	✓	n/a	n/a	n/a
queue.checkin	✓	✓	n/a	n/a	n/a	n/a
queue.priority	✓	✓	✓	n/a	n/a	n/a
consultation.write	n/a	n/a	✓	n/a	n/a	n/a
lab.read	✓	n/a	✓	✓	n/a	n/a
lab.order	n/a	n/a	✓	n/a	n/a	n/a
lab.process	n/a	n/a	n/a	✓	n/a	n/a
pharmacy.read	✓	n/a	✓	n/a	✓	n/a
pharmacy.manage	n/a	n/a	n/a	n/a	✓	n/a
prescriptions.read	✓	n/a	✓	n/a	✓	n/a
prescriptions.dispense	n/a	n/a	n/a	n/a	✓	n/a
beds.read	✓	✓	✓	n/a	n/a	n/a
beds.manage	✓	✓	n/a	n/a	n/a	n/a
admissions.read	✓	✓	✓	n/a	n/a	✓
admissions.write	✓	✓	n/a	n/a	n/a	n/a
admissions.request-discharge	n/a	n/a	✓	n/a	n/a	n/a
admissions.override	✓	n/a	n/a	n/a	n/a	n/a
billing.read	✓	n/a	n/a	n/a	n/a	✓
billing.write	n/a	n/a	n/a	n/a	n/a	✓
reports.read	✓	n/a	n/a	n/a	n/a	n/a
audit.read	✓	n/a	n/a	n/a	n/a	n/a

9. User Interface Design

The interface uses a consistent layout. A role-filtered sidebar, a top bar with global patient search and the user's role, page headers with primary actions, cards, tables and colour-coded status badges (green = available/paid/completed, amber = waiting/pending, red = occupied/unpaid/urgent). Forms validate on the client and show server messages next to the relevant field. The screenshots below were captured automatically by the system tests on the running application.

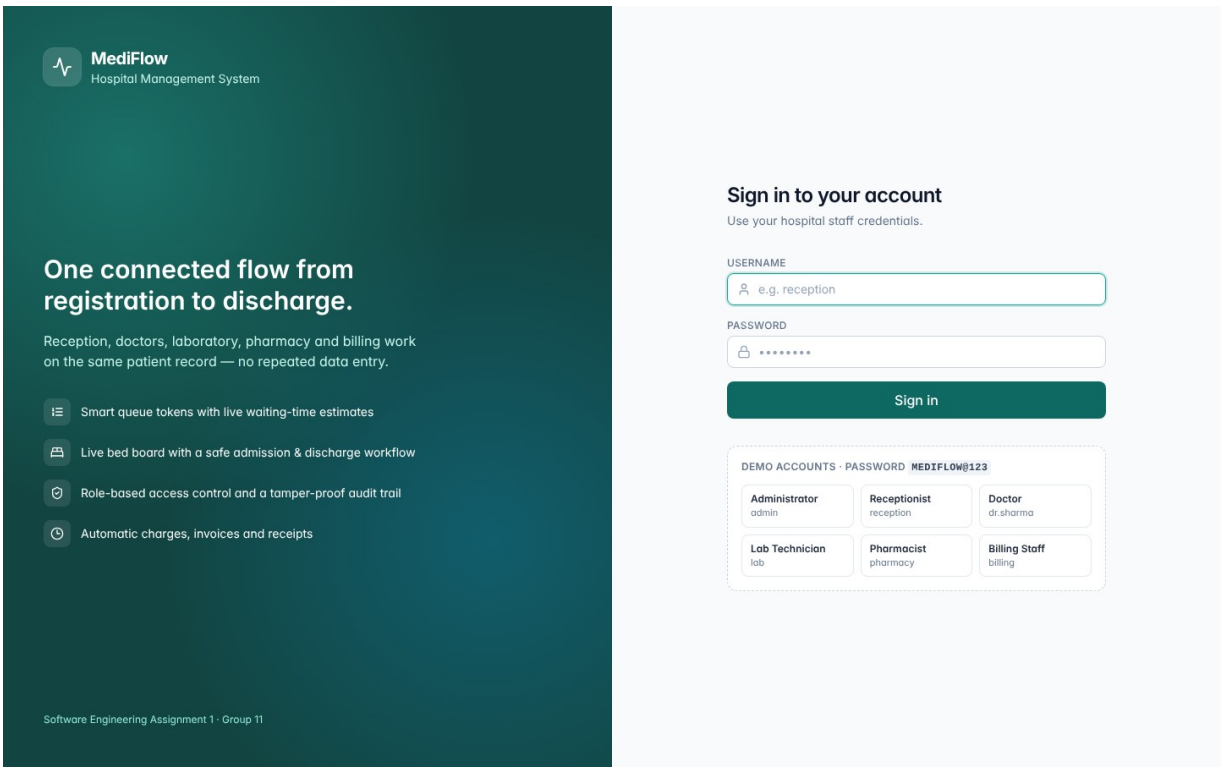


Figure 17. Login screen with demo accounts

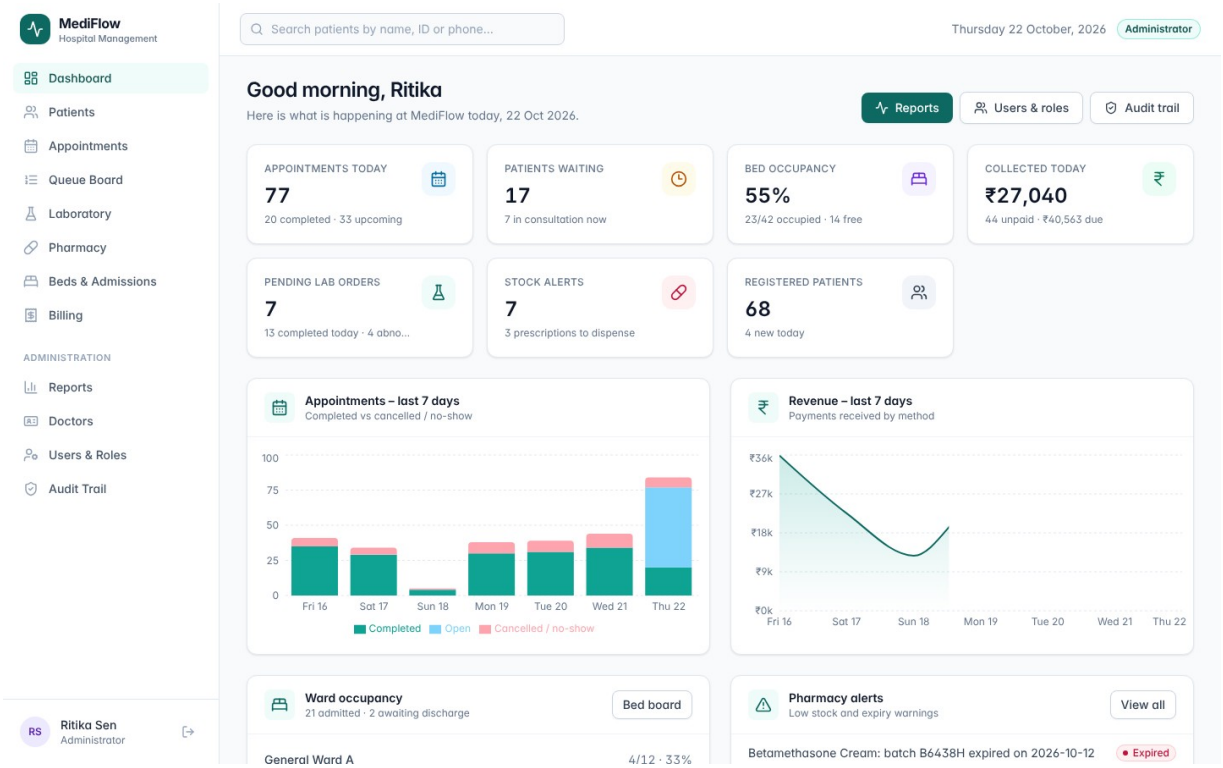


Figure 18. Administrator dashboard (KPIs, 7-day trends, ward occupancy, pharmacy alerts)

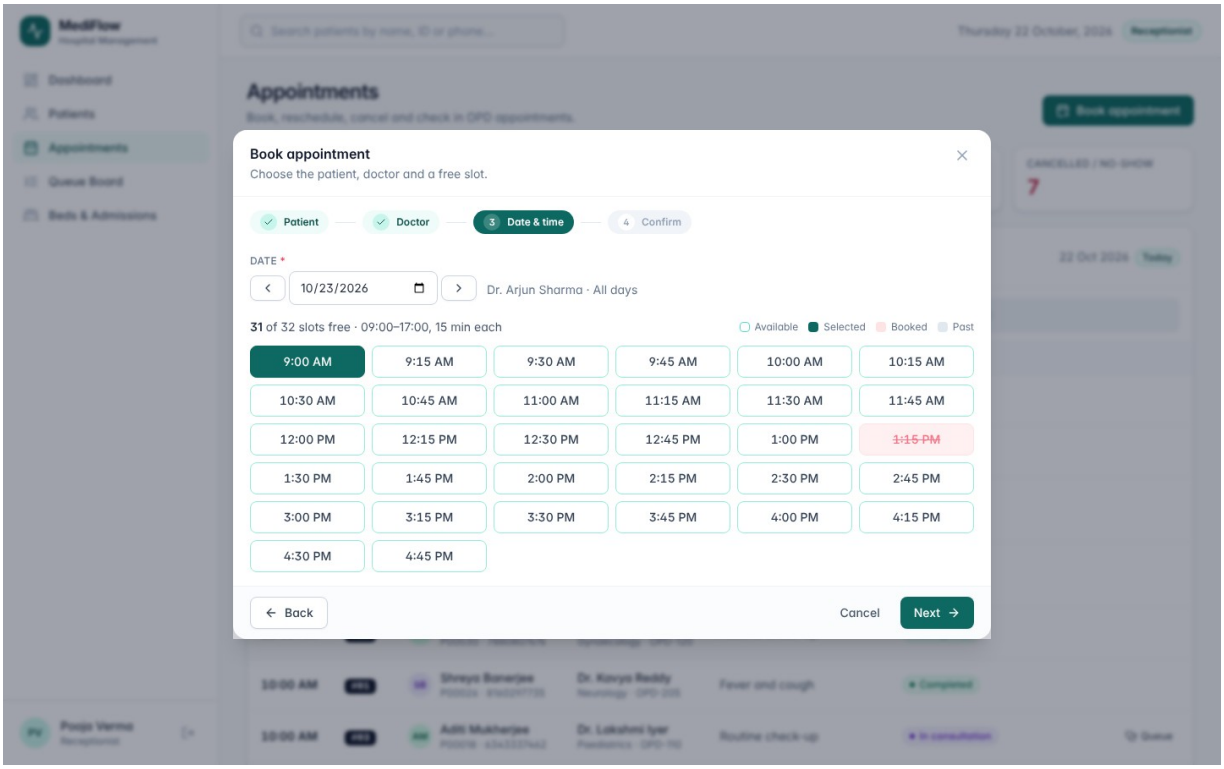


Figure 19. Appointment booking wizard with free / booked / past slots

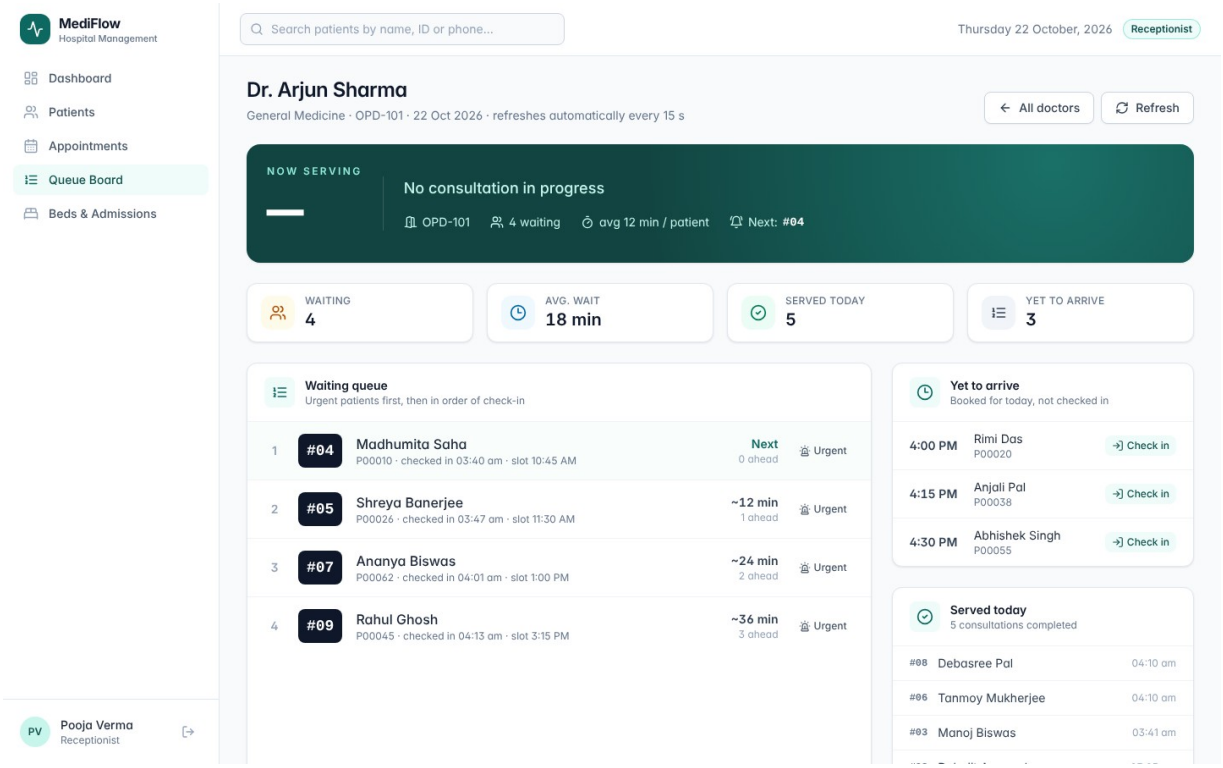


Figure 20. Queue board (now serving, waiting list with tokens, urgent flag and ETA)

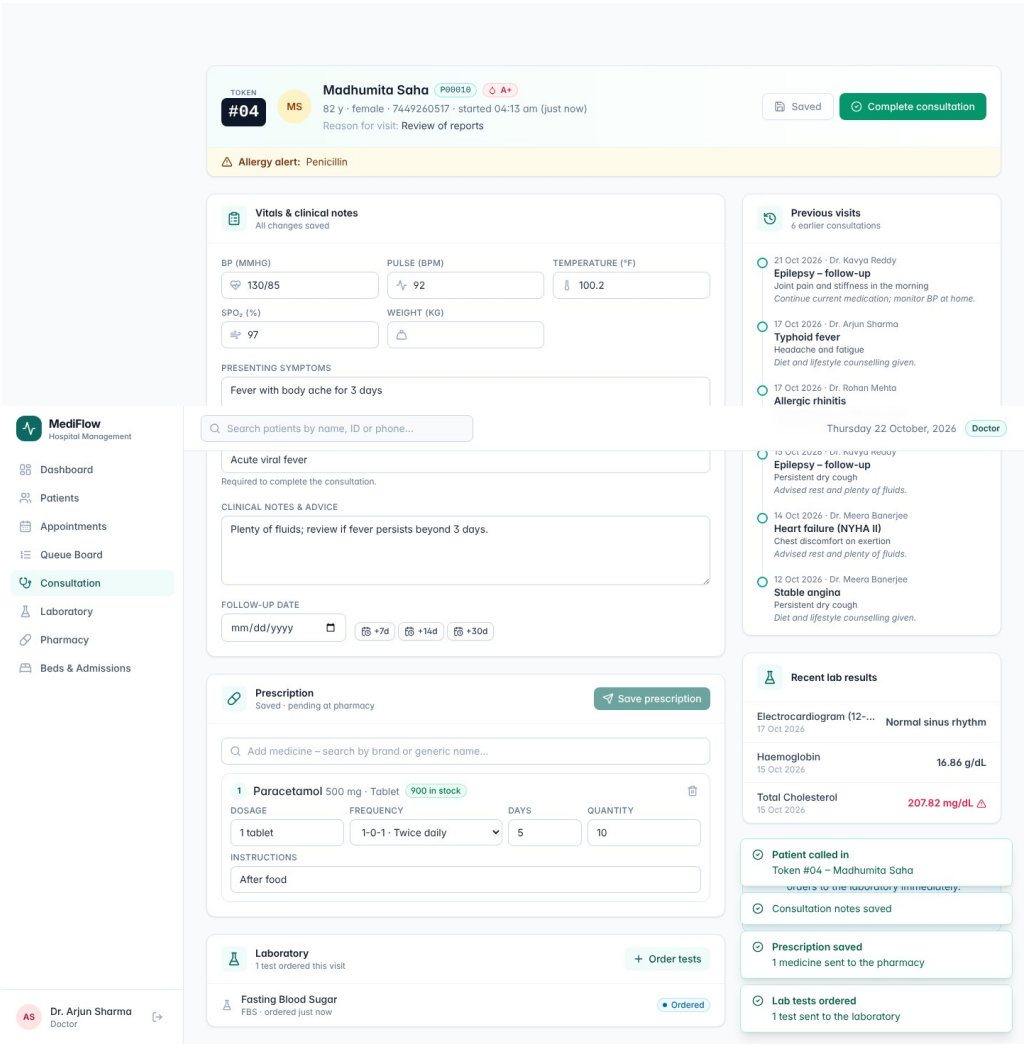


Figure 21. Doctor consultation workspace (vitals, diagnosis, prescription and lab orders)

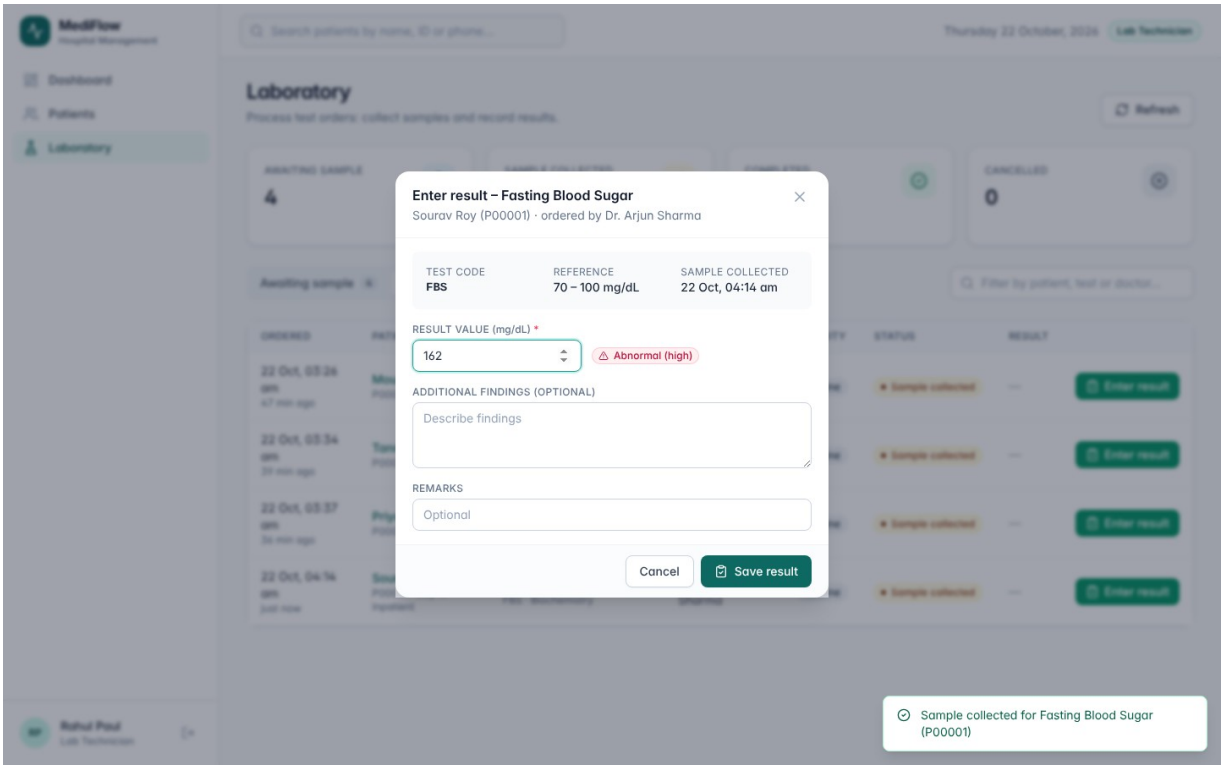


Figure 22. Laboratory result entry with automatic abnormal flag

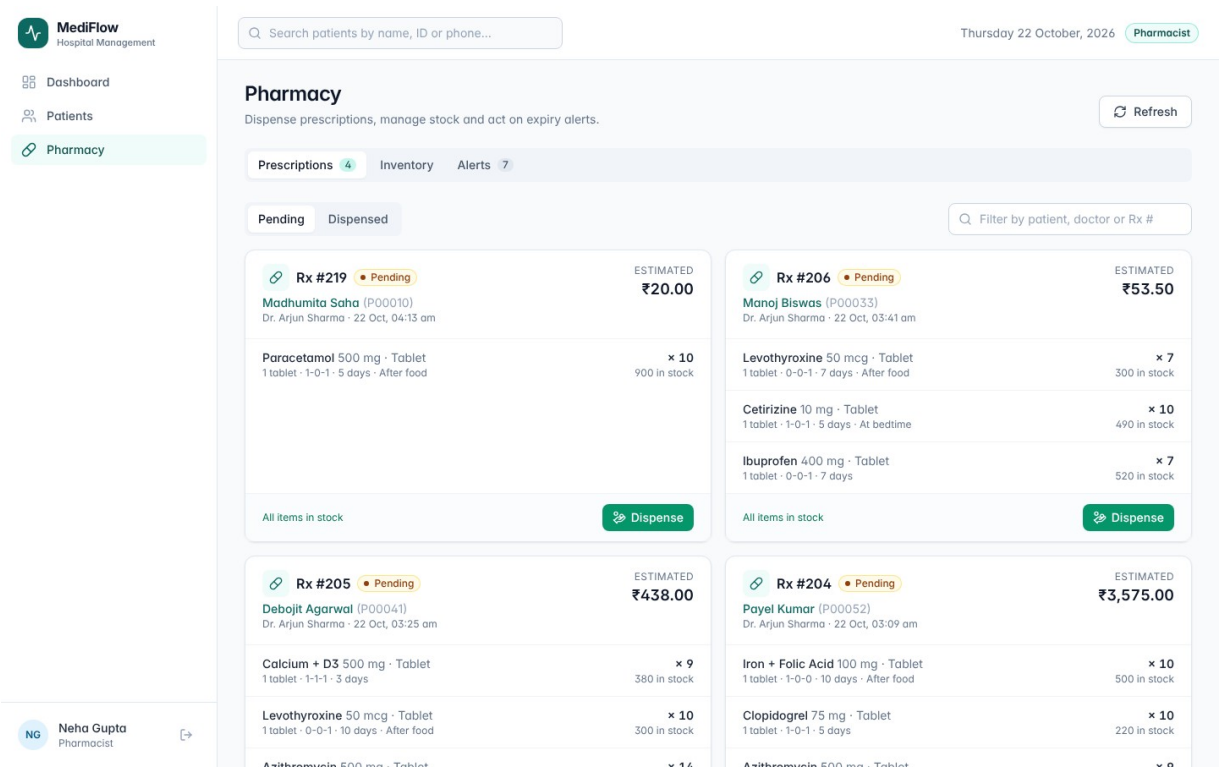


Figure 23. Pharmacy, prescriptions with stock check (undispensable item blocked)

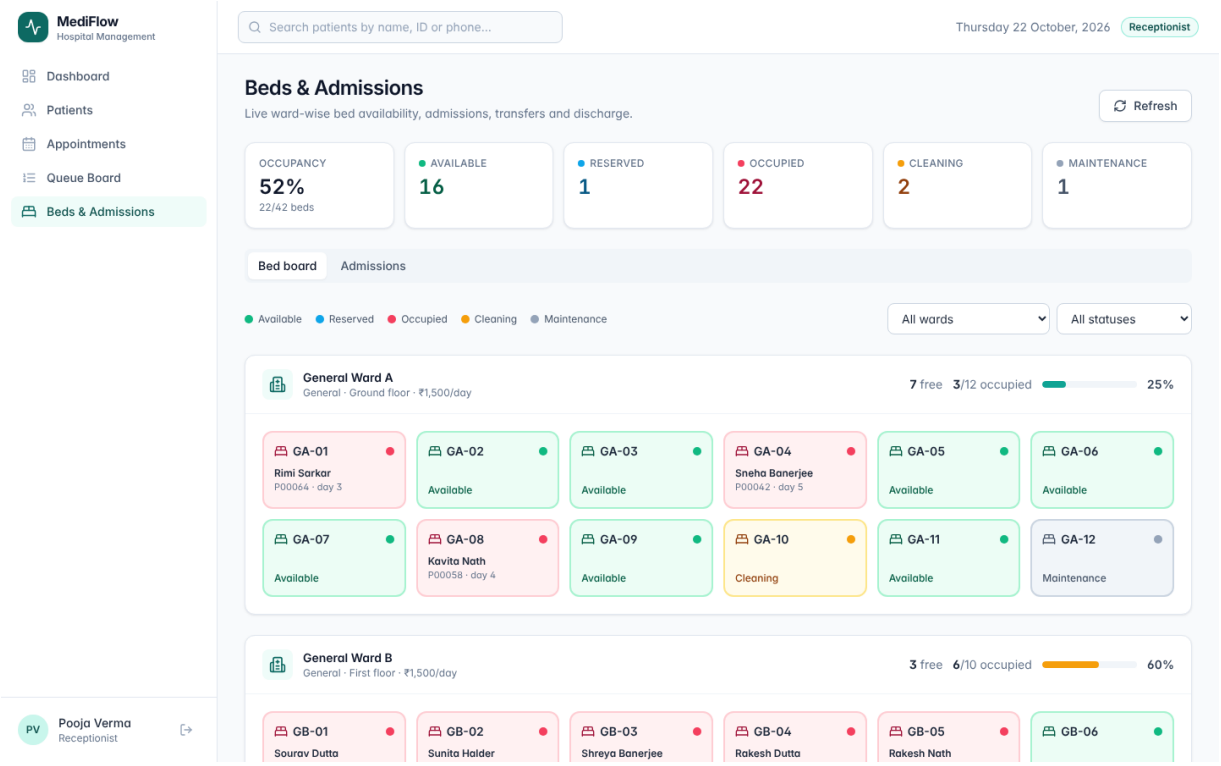


Figure 24. Live bed board colour-coded by status

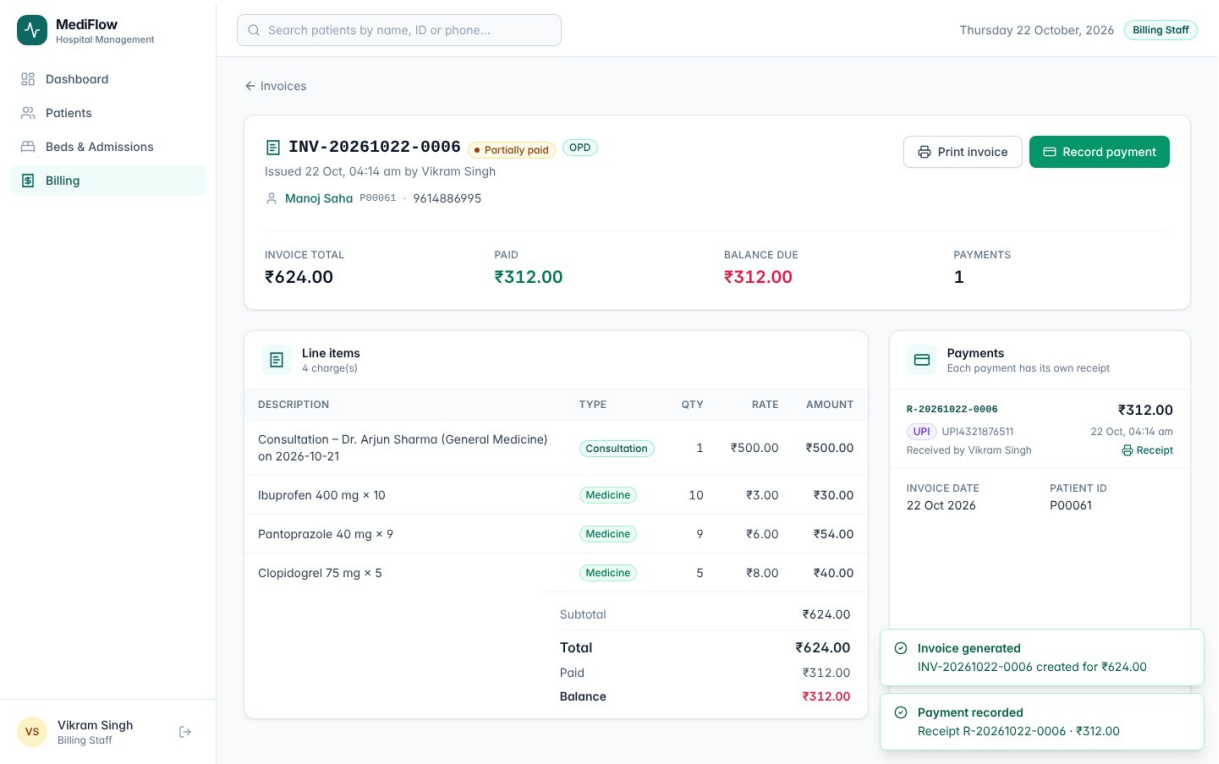


Figure 25. Invoice with line items, discount, payments and balance

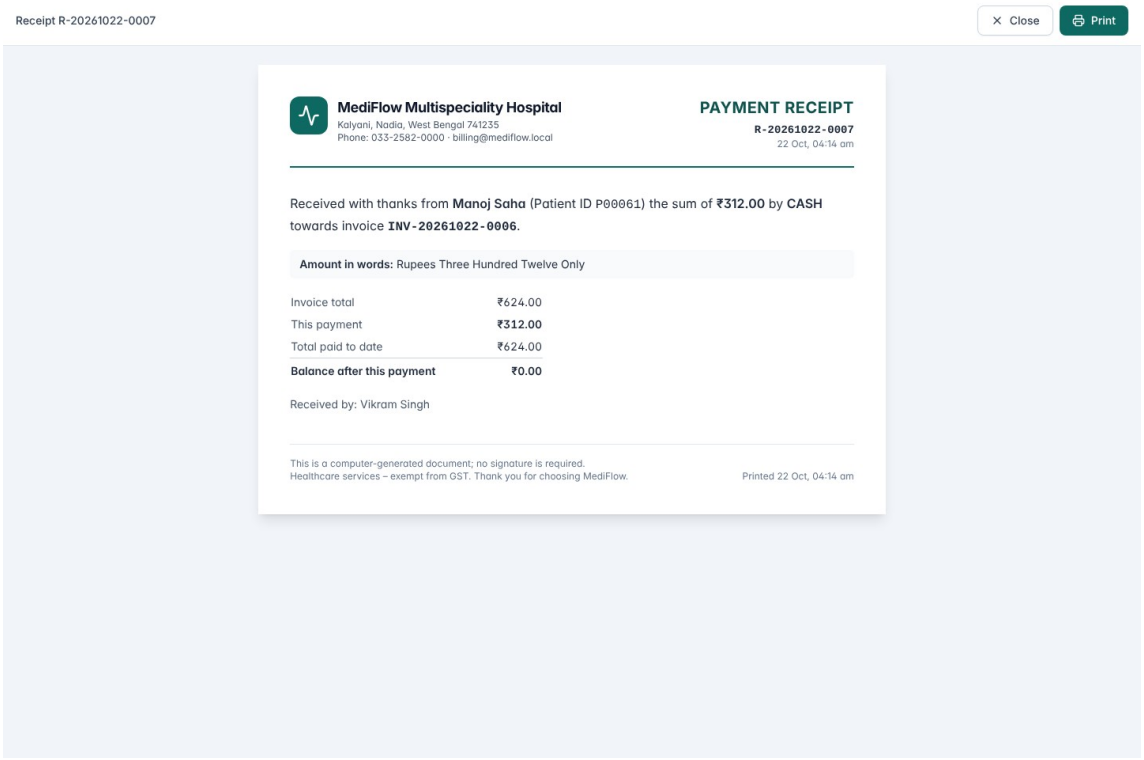


Figure 26. Printable payment receipt

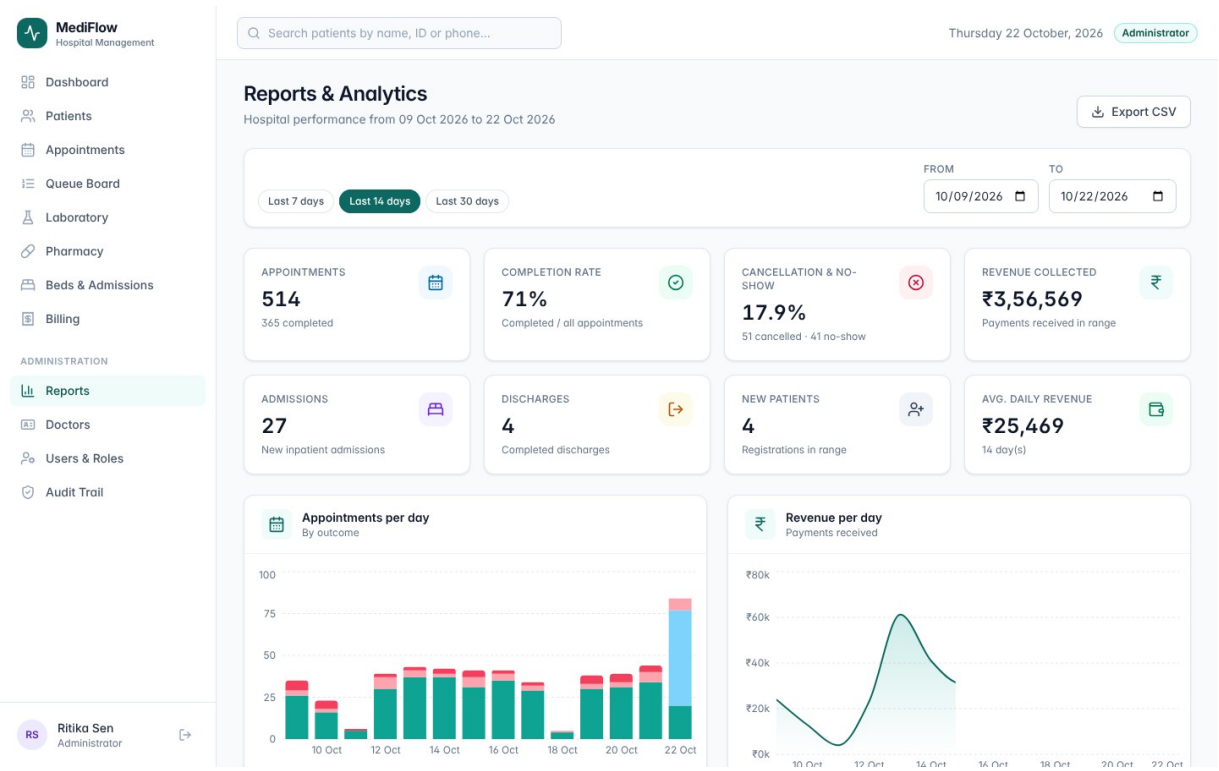


Figure 27. Management reports for a date range

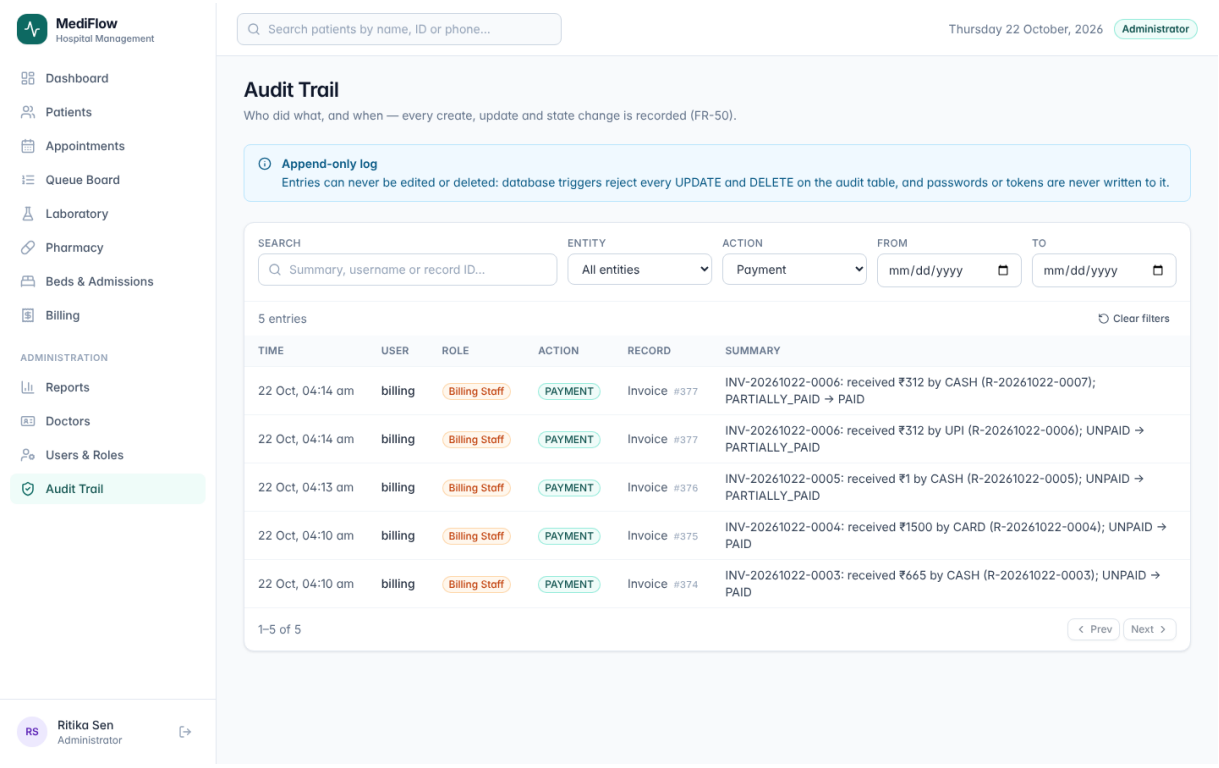


Figure 28. Append-only audit trail

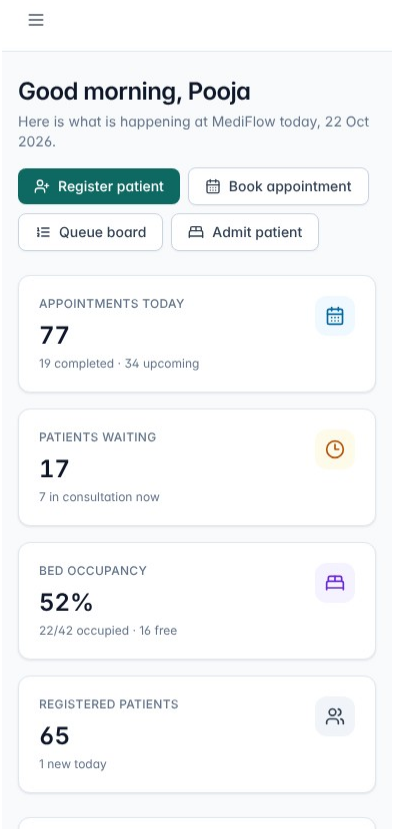


Figure 29. Responsive layout at 390 px (phone)

The module list in this part covers all 50 functional requirements. The traceability matrix in the Software Test Report (Appendix A) traces each one to its module, screen and test cases.